Contents

1 Basic 1
1.1 Default Code 1
1.2 PBDS 1
1.3 int128 Input Output 1
1.4 Python 1
1.5 bitset 2
1.6 離散化 2

2 Math 2
2.1 質數表 2
2.2 快速冪 2
2.3 擴展歐幾里得 2
2.4 矩陣 2
2.5 Miller rabin Prime test . . 3
2.6 Pollard's Rho 3
2.7 皮薩諾定理 3
2.8 高斯消去法 3
2.9 卡特蘭數 4
2.10 中國剩餘定理 4
2.11 歐拉函數 4
2.12 Burnside 引理 4
2.13 Theorem 5
2.14 Estimation 6
2.15 General Purpose Numbers 6

3 Graph 6
3.1 DSU 6
3.2 Dijkstra 6
3.3 SPFA 6
3.4 Floyd Warshell 7
3.5 Tarjan SCC 7
3.6 2 SAT 7
3.7 Euler Path 7
3.8 Bridge 8
3.9 Max flow min cut 8
3.10 Minimum cost maximum flow 9
3.11 二分圖 9
3.12 KM 匈牙利 9
3.13 Check cycle 10
3.14 BCC 10

4 String 10
4.1 trie 10
4.2 KMP 10
4.3 Hash 11
4.4 Zvalue 11
4.5 最長迴文子字串 11
4.6 Suffix Array 11
4.7 AC-Automatan 11
4.8 Duval 12

5 Geometry 12
5.1 Point 12
5.2 內積, 外積, 距離 12
5.3 向量應用 12
5.4 Static Convex Hull 13
5.5 外心, 最小覆蓋圓 13
5.6 最小矩形覆蓋 13
5.7 四邊形旋轉 14
5.8 旋轉 14
5.9 極座標轉直角座標 14
5.10 直角座標轉極座標 14

6 Data Structure 14
6.1 Sparse Table 14
6.2 Segement Tree 15
6.3 Li Chao Tree 16
6.4 動態開點線段樹 16
6.5 Discrete Segement Tree . 17
6.6 Link Cut Tree 18
6.7 BIT 19
6.8 2D BIT 19
6.9 undo DSU 19
6.10 Treap 19
6.11 Treap 區間操作 20

7 Dynamic Programing 21
7.1 LCS 21
7.2 LIS 22
7.3 Knapsack 22
7.4 位元 dp 22
7.5 經典 dp 轉移式 22
7.6 編輯距離 22
7.7 帶權重排程 22

8 Divide and conquer 22
8.1 逆序數對 22
8.2 Mo's algorithm 23

9 Tree 23
9.1 樹直徑 23
9.2 LCA 23
9.3 樹壓平 23
9.4 樹鏈剖分 24

10 Else 25
10.1 Big Number 25
10.2 Tenary Search 25
10.3 對拍 25
10.4 Random Generator 25

1 Basic

1.1 Default Code

```cpp
#include <bits/stdc++.h>
#define int long long
#define endl '\n' // 如果是互動題要把這個註解掉
#define de(x) cout << #x << '=' << x << ", "
#define dd cout << '\n';
// #pragma GCC target("popcnt")
// #pragma GCC optimize("O3")
using namespace std;
int tt = 1;

void pre() {
  cout.tie(nullptr); // 輸出加速
  cin >> tt; // 多筆輸入
}

void solve() {}

signed main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr);
#ifdef LOCAL
  // g++ -DLOCAL -std=c++17 <filename> && ./a.out
  freopen("input.txt", "r", stdin);
  // freopen("output.txt", "w", stdout);
#endif // LOCAL
  pre();
  while (tt--) { solve(); }
  return 0;
}
```

1.2 PBDS

```cpp
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
using namespace std;

template
     <class T> using Tree = tree<T, null_type, less<T
    >, rb_tree_tag, tree_order_statistics_node_update>;

/*
如果有 define int long long 記得拿掉
Tree<int> t 就跟 set<int> t 一樣，有包好 template
rb_tree_tag 使用紅黑樹
第三個參數 less<T> 為由小到大，greater<T> 為由大到小
插入 t.insert(); 刪除 t.erase();
t.order_of_key
    (k); 從前往後數 k 是第幾個 (0-base 且回傳 int 型別)
t.find_by_order(k);
    從前往後數第 k 個元素 (0-base 且回傳 iterator 型別)
t.lower_bound
    (); t.upper_bound(); 用起來一樣 回傳 iterator
可以用 Tree<pair<int, int>> T 來模擬 mutiset
*/
```

1.3 int128 Input Output

```cpp
// 抄 BBuf github 的

#include <bits/stdc++.h>
using namespace std;

// 輸入須為 int128 範圍內的整數；可有空白、正負號。
// EOF 回 false；使用 getchar，勿混用關閉同步的 cin。
bool scan(__int128 &x) { // 輸入
  int ch = getchar();
  while (ch != EOF && isspace((unsigned char)ch))
    ch = getchar();
  if (ch == EOF) return false;
  bool neg = ch == '-';
  if (ch == '-' || ch == '+') ch = getchar();
  __uint128_t v = 0;
  while (ch >= '0' && ch <= '9') {
    v = v * 10 + ch - '0';
    ch = getchar();
  }
  x = neg && v ? -(__int128)(v - 1) - 1 : (__int128)v;
  return true;
}

void print(__int128 x) { // 輸出
  if (x < 0) putchar('-');
  __uint128_t v = x;
  if (x < 0) v = 0 - v;
  char s[40];
  int n = 0;
  do { s[n++] = v % 10 + '0'; v /= 10; } while (v);
  while (n) putchar(s[--n]);
}

int main() {
  __int128 a, b;
  if (!scan(a) || !scan(b)) return 0;
  // 範例：加法、乘法的結果仍須放得進 int128。
  print(a + b);
  puts("");
  print(a * b);
  return 0;
}
```

1.4 Python

```python
# Python 3；區塊結尾用冒號、縮排 4 格，不用大括號
import sys
# 調高遞迴上限；很深的 DFS 宜改迭代
sys.setrecursionlimit(20000)
input = sys.stdin.readline  # 保留換行；EOF 回傳 ""

n, m = map(int, input().split())  # 同一行兩個整數
a = list(map(int, input().split()))  # 一行整數
a = list(map(float, input().split()))  # 一行浮點數
g = [list(map(int, input().split())) for _ in range(n)]
s = input().rstrip("\r\n")  # 字串；保留頭尾空白
for line in sys.stdin:  # 一直讀到 EOF
```

```
    a = list(map(int, line.split()))
# 多筆：先讀 t = int(input())，再 for _ in range(t):
print(*a)  # 空白分開；print(x, end="") 不換行
print("\n".join(map(str, a)))  # 每行一個
print(f"{3.14159:.3f}")  # 小數點後 3 位

# list / 迴圈 / 排序（索引從 0 開始）
# range(l, r) 不含 r；倒序 range(n-1, -1, -1)
for i, x in enumerate(a): print(i, x)  # 索引、值
a.append(5); x = a.pop()  # push_back / 取出尾端
b = a[1:3]; c = a[::-1]  # 切片不含右端；反轉副本
a.sort()  # 原地排序，回傳 None；勿寫 a = a.sort()
b = sorted(a, reverse=True)  # 新陣列，遞減排序
d = {}; d[x] = d.get(x, 0) + 1  # dict 計數
seen = set(a); print(x in seen)  # 去重、判斷存在

# 複製 / reference：list 可修改，int / str 不可修改
a = [1, 2]; b = a  # 共用同一物件，不會複製
# 此時 b[0] = 9 會改到 a；b = [9] 則只是換綁定
b = a[:]  # 或 a.copy()；只複製外層，內層仍共用
g = [[0] * m for _ in range(n)]  # 勿用 [[0]*m]*n
h = [row[:] for row in g]  # 二維數字陣列逐列複製
from copy import deepcopy
h = deepcopy(g)  # 更深的巢狀結構也獨立複製
def change(a):  # 傳入 list 也共用物件
    a.append(1)  # 會改外部 list；改寫 a = [] 則不會

# 值比較用 ==；is 是同一物件（None 判斷用 is None）
# / 是浮點除法；// 向下取整：-7 // 3 == -3
# -7 % 3 == 2；次方用 **，^ 是 XOR
# map 是一次性迭代器；要索引或重複使用先包 list

# Fraction(分子, 分母)：精確分數，可直接加減乘除
from fractions import Fraction
frac = Fraction(3, 4)
numerator = frac.numerator  # 取出分子
denominator = frac.denominator  # 取出分母
print(frac + Fraction(1, 6))  # 11/12
# 精確的 1/10 用 Fraction(1, 10)，不要 Fraction(0.1)
```

1.5 bitset

```
bitset<size> b(a):長度為size，初始化為a
b[i]:第i位元的值(0 or 1)
b.size():有幾個位元
b.count():有幾個1
b.set():所有位元設為1
b.reset():所有位元設為0
b.flip():所有位元反轉
```

1.6 離散化

```
// 建表 O(n log n)，查詢 O(log n)，不修改原陣列
// 編號從 1 開始；查詢值必須已收集
struct Discretize {
  vector<int> xs;

  Discretize(const vector<int> &a) : xs(a) {
    sort(xs.begin(), xs.end());
    xs.erase(unique(xs.begin(), xs.end()), xs.end());
  }

  int operator()(int x) const {
    auto it = lower_bound(xs.begin(), xs.end(), x);
    assert(it != xs.end() && *it == x);
    return it - xs.begin() + 1;
  }

  int size() const { return xs.size(); }
};

// Discretize d({30, 10, 30});
// d(10) -> 1, d(30) -> 2
```

2 Math

2.1 質數表

```
vector<int> prime_table(int n){
  vector<int> table(n + 1, 0);
  for(int i = 1; i <= n; i++){
    for(int j = i; j <= n; j += i){
      table[j]++;
    }
  }
  return table;
}
```

2.2 快速冪

```
#define int long long

// 根據費馬小定
    理，若 a p 互質，a^(p-2) 為 a 在 mod p 時的乘法逆元
// 上式還需 p 為質數。
// a ^ (b ^ c
    ) % mod = fast_pow(a, fast_pow(b, c, mod - 1), mod)
// 上式需 mod 為質數且 a 與 mod 互質。
typedef unsigned long long ull;
inline int ksc(ull x, ull y, int p) {
  // O(1)快速乘（防爆long long）；p > 0，GCC/Clang。
  return (__uint128_t)x * y % p;
}

// a^b % mod；b >= 0、mod > 0，支援負底數。
inline int fast_pow(int a, int b, int mod) {
  a %= mod;
  if (a < 0) a += mod;
  int res = 1 % mod;
  for (; b; b >>= 1, a = ksc(a, a, mod))
    if (b & 1) res = ksc(res, a, mod);
  return res;
}
```

2.3 擴展歐幾里得

```
int gcd(int a, int b)
{
  return b == 0 ? a : gcd(b, a % b);
}

int lcm(int a, int b)
{
  return a * b / gcd(a, b);
}

pair<int, int> ext_gcd
    (int a, int b) //擴展歐幾里德 ax+by = gcd(a,b)
{
  if (b == 0)
    return {1, 0};
  if (a == 0)
    return {0, 1};
  int x, y;
  tie(x, y) = ext_gcd(b % a, a);
  return make_pair(y - (b / a) * x, x);
}
```

2.4 矩陣

```
template<typename T>
struct Matrix{
  using rt = std::vector<T>;
  using mt = std::vector<rt>;
  using matrix = Matrix<T>;
  int r,c;
  mt m;
  Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
  rt& operator[](int i){return m[i];}
  matrix operator+(const matrix &a){
    matrix rev(r,c);
    for(int i=0;i<r;++i)
      for(int j=0;j<c;++j)
        rev[i][j]=m[i][j]+a.m[i][j];
    return rev;
  }
  matrix operator-(const matrix &a){
    matrix rev(r,c);
    for(int i=0;i<r;++i)
      for(int j=0;j<c;++j)
        rev[i][j]=m[i][j]-a.m[i][j];
    return rev;
  }
```

```
  matrix operator*(const matrix &a){
    matrix rev(r,a.c);
    matrix tmp(a.c,a.r);
    for(int i=0;i<a.r;++i)
      for(int j=0;j<a.c;++j)
        tmp[j][i]=a.m[i][j];
    for(int i=0;i<r;++i)
      for(int j=0;j<a.c;++j)
        for(int k=0;k<c;++k)
          rev.m[i][j]+=m[i][k]*tmp[j][k];
    return rev;
  }
  bool inverse(){
    Matrix t(r,r+c);
    for(int y=0;y<r;y++){
      t.m[y][c+y] = 1;
      for(int x=0;x<c;++x)
        t.m[y][x]=m[y][x];
    }
    if( !t.gas() )
      return false;
    for(int y=0;y<r;y++)
      for(int x=0;x<c;++x)
        m[y][x]=t.m[y][c+x]/t.m[y][y];
    return true;
  }
  T gas(){
    vector<T> lazy(r,1);
    bool sign=false;
    for(int i=0;i<r;++i){
      if( m[i][i]==0 ){
        int j=i+1;
        while(j<r&&!m[j][i])j++;
        if(j==r)continue;
        m[i].swap(m[j]);
        sign=!sign;
      }
      for(int j=0;j<r;++j){
        if(i==j)continue;
        lazy[j]=lazy[j]*m[i][i];
        T mx=m[j][i];
        for(int k=0;k<c;++k)
          m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx;
      }
    }
    T det=sign?-1:1;
    for(int i=0;i<r;++i){
      det = det*m[i][i];
      det = det/lazy[i];
      for(auto &j:m[i])j/=lazy[i];
    }
    return det;
  }
};
```

2.5 Miller rabin Prime test

```
// fast_pow 去前面抄，需要處裡防暴乘法
// 記得 #define int long long 也要放
// 配合 ksc 的 __uint128_t 版本，適用正 long long。
// time: O(logn)，每個底數只做一次快速冪。
// 底數來源：https://miller-rabin.appspot.com/

// p > 2 為奇數，x % p != 0。
inline bool mr(int x, int p) {
  int d = p - 1, s = 0;
  while (!(d & 1)) d >>= 1, ++s;
  int z = fast_pow(x % p, d, p);
  if (z == 1 || z == p - 1) return 1;
  for (int i = 1; i < s; ++i) {
    z = ksc(z, z, p);
    if (z == p - 1) return 1;
  }
  return 0;
}

inline bool prime(int x) {
  if (x < 2 || x % 2 == 0) return x == 2;
  // 這 7 個底數可判定小於 2^64 的整數。
  for (int a : {2LL, 325LL, 9375LL, 28178LL,
                450775LL, 9780504LL, 1795265022LL})
    if (a % x && !mr(a, x)) return 0;
  return 1;
}
```

2.6 Pollard's Rho

```
// 主函數記得放 srand(time(nullptr))
// prime 檢測以及快速冪, gcd 等請從前面抄

// 輸入一個數字 p，隨
    機回傳一個 非 1 非 p 的因數，若 p 是質數會無窮迴圈
#define rg register int
inline int rho(int p) {
  int x, y, z, c, g;
  rg i, j;
  while (1) {
    y = x = rand() % p;
    z = 1;
    c = rand() % p;
    i = 0, j = 1;
    while (++i) {
      x = (ksc(x, x, p) + c) % p;
      z = ksc(z, abs(y - x), p);
      if (x == y || !z) break;
      if (!(i % 127) || i == j) {
        g = gcd(z, p);
        if (g > 1) return g;
        if (i == j) y = x, j <<= 1;
      }
    }
  }
}

// 回傳隨機一個質因數，若 input 為質數，則直接回傳
int prho(int p){
  if(prime(p)) return p;
  int m = rho(p);
  if(prime(m)) return m;
  return prho(p / m);
}

// 回傳將 n 質因數分解的結果，由小到大排序
// ex: input: 48, output: 2 2 2 2 3
vector<int> prime_factorization(int n){
  vector<int> ans;
  while(n != 1){
    int m = prho(n);
    ans.push_back(m);
    n /= m;
  }
  sort(ans.begin(), ans.end());
  return ans;
}
```

2.7 皮薩諾定理

```
// fib(x) % m = fib(x + kn) % m 當 k >= 1，求 n
// n 為費式數列 % m 會重複的週期
// pisano_period(m) <= 6m
// 通常這都要本地跑

#define int long long

// m >= 1，6m 須放得進 long long；O(週期)。
int pisano_period(int m) {
  if (m == 1) return 1;
  int pre = 0, cur = 1;
  for (int i = 1;; ++i) {
    int next = (pre + cur) % m;
    pre = cur, cur = next;
    if (pre == 0 && cur == 1) return i;
  }
}
```

2.8 高斯消去法

```
from fractions import Fraction
def gauss_elimination(matrix, results):
    # 將所有數字轉換為分數
    n = len(matrix)
    augm = [[Fraction(matrix
        [i][j]) for j in range(n)] for i in range(n)]
    augr = [Fraction(results[i]) for i in range(n)]

    # 高斯消去法
    for i in range(n):
        # 尋找主元
        if augm[i][i] == 0:
```

```
            for j in range(i + 1, n):
                if augm[j][i] != 0:
                    augm[i], augm[j] = augm[j], augm[i]
                    augr[i], augr[j] = augr[j], augr[i]
                    break

        pivot = augm[i][i]
        if pivot == 0:
            # 如果主元為0，繼續檢查該行是否全為 0
            if all(augm[i][j] == 0 for j in range(n)):
                if augr[i] != 0:
                    return None  # 無解
                else:
                    continue
                        # 可能有無限多解，繼續檢查

        # 將主元行的數字規一化
        for j in range(i, n):
            augm[i][j] /= pivot
        augr[i] /= pivot

        # 將其他行的數字變為0
        for j in range(n):
            if i != j:
                factor = augm[j][i]
                for k in range(i, n):
                    augm[j][k] -= factor * augm[i][k]
                augr[j] -= factor * augr[i]
    # 檢查是否存在無限多解的情況
    for i in range(n):
        if all(augm[i][j
            ] == 0 for j in range(n)) and augr[i] == 0:
            return [] # 無限多組解

    return augr


# matrix = [
#     [2, -1, 1],
#     [3, 3, 9],
#     [3, 3, 5]
# ]
# results = [8, -42, 0]
# output = [
    Fraction(12, 1), Fraction(11, 2), Fraction(-21, 2)]
# Fraction 可以強轉 float


import numpy as np

def gauss_elimination(matrix, ans):
    matrix = np.array(matrix)
    ans = np.array(ans)

    try:
        solution = np.linalg.solve(matrix, ans)
        return [f"{value:.2f}" for value in solution]
    except np.linalg.LinAlgError:
        # 無解或者無限多組解
        return "No Solution"

# 有開放 numpy 可以用
# 優點: 行數短, 執行速度快
# 缺點: 只能用浮點數, 無法區分無解及無限多組解
```

2.9 卡特蘭數

```
"""
卡特蘭數 Catalan
公式：H(n) = C(2 * n, n) // (n + 1), n >= 2, n 為正整數
快速計算方式：
1. H(0) = H(1) = 1, H(n)
    = sum(H(i - 1) * H(n - i) for i in range(1, n + 1))
2. H(n) = H(n - 1) * (4 * n - 2) // (n + 1)
3. H(n) = C(2 * n, n) - C(2 * n, n - 1)
"""

"""
可解問題：
有效括號匹配問題：
    給定 n 個左括號與右括號，求有幾種不同的正確括號匹配
二元樹結構問題：給定 n 個節點，求有幾種不同的二元樹結構
將一個凸
    n + 2 邊形劃分成多個三角形，求有幾種不同的劃分方式
狄克路徑：給定 n * n 的網格，
    從左下到右上的路徑中，永不超過對角線的路徑有幾種
一個 stack 在 push 順
    序不變的情況下 (1, 2, 3, ..., n)，有幾種 pop 的方式
在圖上選擇 2 * n 個
    點，將這些點兩兩連接使得 n 條線段不相交的方法有幾種
"""

n = int(input())

catalan = [1 for _ in range(n + 1)]
for i in range(1, n + 1):
    catalan
        [i] = catalan[i - 1] * (4 * i - 2) // (i + 1)

ans = 0
for i in range(0, n + 1):  # 卡特蘭數的平方
    ans += catalan[i] * catalan[n - i]

print(ans)
# 185ms in codeforces, n <= 5000
```

2.10 中國剩餘定理

```
// vec[i] = {m_i, x_i}，求最小非負 x
    使得 x ≡ x_i (mod m_i) 對所有 i 同時成立；無解回 -1
// 注意 overflow
int CRT(vector<pair<int, int>> &v)
{
  int m = v[0].first, x = (v[0].second % m + m) % m;
  for (int i = 1; i < (int)v.size(); ++i)
  {
    int mi =
        v[i].first, xi = (v[i].second % mi + mi) % mi;
    int g = gcd(m, mi), d = xi - x;
    if (d % g) return -1;
    int m1 = m / g, m2 = mi / g;
    auto ab = ext_gcd((int)m1, (int)m2);
    int inv = ((int)ab.first % m2 + m2) % m2;
    int k = ((d / g) % m2 + m2) % m2;
    k = (k * inv) % m2;
    x = (x + m * k) % (m * m2);
    m *= m2;
    x = (x + m) % m;
  }
  return x;
}
```

2.11 歐拉函數

```
#define int long long

// 歐拉函數 φ(n)：1..n 中與 n 互質的整數個數
// φ(n) = n * Π_{p|n} (1 - 1/p)
int phi(int n) {
  int r = n;
  for (int p = 2; p * p <= n; ++p)
    if (n % p == 0) {
      r -= r / p;
      while (n % p == 0) n /= p;
    }
  if (n > 1) r -= r / n;
  return r;
}
```

2.12 Burnside 引理

```
#define int long long

// Burnside 引理：軌道數 = (1/|G|) Σ_{g∈G} |Fix(g)|
// 著色時 |Fix(g)| = k^(g 的循環個數)
// 項鍊 = 只算旋轉（循環群 C_n）
// 手鍊 = 旋轉 + 翻轉（二面體群 D_n）
// fast_pow
    去前面抄（快速冪），phi 去前面抄（歐拉函數）
// mod 需為質數（用費馬小定理求逆元）

// 項鍊：n 顆珠子、k 色，本質不同數量
// = (1/n) Σ_{d|n} φ(d) * k^(n/d)
int necklace(int n, int k, int mod) {
  int sum = 0;
```

```
  for (int d = 1; d * d <= n; ++d)
    if (n % d == 0) {
      sum = (sum + phi(d) % mod *
                   fast_pow(k, n / d, mod)) % mod;
      if (d * d != n)
        sum = (sum + phi(n / d) % mod *
                     fast_pow(k, d, mod)) % mod;
    }
  return sum * fast_pow(n, mod - 2, mod) % mod;
}

// 手鍊：旋轉 + 翻轉
// = (1/2n) [ Σ_{d|n} φ(d) k^(n/d) + R ]
//   n 奇: R = n * k^((n+1)/2)
//   n 偶: R = (n/2) * k^(n/2) + (n/2) * k^(n/2+1)
int bracelet(int n, int k, int mod) {
  int rot
      = necklace(n, k, mod) * n % mod;  // 旋轉部分總和
  int refl;
  if (n & 1)
    refl = n % mod *
           fast_pow(k, (n + 1) / 2, mod) % mod;
  else
    refl = (n / 2) % mod *
           (fast_pow(k, n / 2, mod) +
            fast_pow(k, n / 2 + 1, mod)) % mod;
  return (rot + refl) % mod *
         fast_pow(2 * n, mod - 2, mod) % mod;
}
```

2.13 Theorem

- Cramer 法則

$$\begin{matrix} ax+by=e \\ cx+dy=f \end{matrix} \Rightarrow \begin{matrix} x=\dfrac{ed-bf}{ad-bc} \\ y=\dfrac{af-ec}{ad-bc} \end{matrix}$$

- Vandermonde 恆等式

$$C(n+m,k)=\sum_{i=0}^{k} C(n,i)C(m,k-i)$$

- 隔板法＋排容原理（容斥原理）
  設整數 $n\geq 0$、$k\geq 1$，將 n 個相同物品分給 k 個不同對象。約定 $\binom{a}{b}=0$ 若 $a<0$、$b<0$ 或 $b>a$。

$$x_1+\cdots+x_k=n, x_i\geq 0 : \binom{n+k-1}{k-1},$$

$$x_1+\cdots+x_k=n, x_i\geq 1 : \binom{n-1}{k-1}.$$

  若有整數下界 $x_i\geq a_i$，先令 $y_i=x_i-a_i$，總和改為 $n-\sum_i a_i$。排容原理：扣掉不合法情況，再補回重複扣除的交集。

$$\left|\bigcup_{i=1}^{k} A_i\right| = \sum_{\emptyset\neq S\subseteq\{1,\ldots,k\}} (-1)^{|S|+1} \left|\bigcap_{i\in S} A_i\right|.$$

  若限制 $0\leq x_i\leq b_i$（b_i 為非負整數），解數為

$$\sum_{S\subseteq\{1,\ldots,k\}} (-1)^{|S|} \binom{n-\sum_{i\in S}(b_i+1)+k-1}{k-1}.$$

  S 表示指定違反上界的變數；枚舉 2^k 個子集合，適合 k 小時使用。
- Lucas 定理
  設 p 為質數、n, k 為非負整數，將 n, k 寫成 p 進位：$n=\sum_i n_i p^i$、$k=\sum_i k_i p^i$，則

$$\binom{n}{k}\equiv\prod_i \binom{n_i}{k_i} \pmod{p}.$$

  實作時反覆取 $n \bmod p$、$k \bmod p$，再各自除以 p；任一位 $k_i>n_i$ 時答案為 0。p 較小時，可用 $O(p)$ 預處理 0 到 $p-1$ 的階乘與反階乘，每次查詢只需逐位相乘；不直接適用於合數模數。
- Legendre 公式
  設 p 為質數、$n\geq 0$，$n!$ 中質因數 p 的次方為

$$\nu_p(n!)=\sum_{j\geq 1}\left\lfloor\frac{n}{p^j}\right\rfloor.$$

  實作時令 $t=n$，反覆令 $t=\lfloor t/p\rfloor$ 並累加，直到 $t=0$；避免直接計算 p^j 溢位。組合數的次方為 $\nu_p\binom{n}{k}=\nu_p(n!)-\nu_p(k!)-\nu_p((n-k)!)$ $(0\leq k\leq n)$；$n!$ 的十進位尾零數為 $\nu_5(n!)$。
- 錯排公式
  D_n 為 n 個不同元素的排列中，沒有任何元素留在原位的方案數。$D_0=1$、$D_1=0$，且

$$D_n=(n-1)(D_{n-1}+D_{n-2}) \quad (n\geq 2).$$

  用前兩項遞推，時間 $O(n)$、額外空間 $O(1)$。可逐步取模；加法與乘法仍須防溢位。
- Kirchhoff 定理
  設 L 為圖 G 的 $n\times n$ Laplacian 矩陣，其中 $L_{ii}=d(i)$，$L_{ij}=-c$，c 為邊 (i,j) 的條數。
  - 無向圖的生成樹數為 $|\det(\tilde{L}_{11})|$。
  - 以 r 為根的有向生成樹數為 $|\det(\tilde{L}_{rr})|$。
- Tutte 矩陣
  令 D 為 $n\times n$ 矩陣，當 $i<j$ 且 $(i,j)\in E$ 時令 $d_{ij}=x_{ij}$（x_{ij} 由均勻隨機選取），否則令 $d_{ij}=-d_{ji}$。則 $\dfrac{\text{rank}(D)}{2}$ 為圖 G 的最大匹配數。
- Cayley 公式
  - 對於帶標號頂點、度數序列為 $d_1, d_2, \ldots, d_n$ 的情形，生成樹數為 $\dfrac{(n-2)!}{(d_1-1)!(d_2-1)!\cdots(d_n-1)!}$。
  - 設 $T_{n,k}$ 為在 n 頂點、k 個連通分量的帶標號森林數，且頂點 $1,2,\ldots,k$ 分屬不同分量，則 $T_{n,k}=kn^{n-k-1}$。
- Erdős–Gallai 定理
  非負整數序列 $d_1\geq\cdots\geq d_n$ 可表示為 n 頂點簡單圖的度數序列，當且僅當 $d_1+\cdots+d_n$ 為偶數，且對每個 $1\leq k\leq n$ 皆有

$$\sum_{i=1}^{k} d_i\leq k(k-1)+\sum_{i=k+1}^{n}\min(d_i,k).$$

- Gale–Ryser 定理
  非負整數序列 $a_1\geq\cdots\geq a_n$ 與 $b_1,\ldots,b_n$ 構成雙圖度數序列（bigraphic）的充要條件為

$$\sum_{i=1}^{n} a_i=\sum_{i=1}^{n} b_i \quad \text{且} \quad \sum_{i=1}^{k} a_i\leq\sum_{i=1}^{n}\min(b_i,k) \text{ 對所有} 1\leq k\leq n.$$

- Fulkerson–Chen–Anstee 定理
  一序列 $(a_1,b_1),\ldots,(a_n,b_n)$（非負整數對，且 $a_1\geq\cdots\geq a_n$）為有向圖可實現之入出度序列，當且僅當

$$\sum_{i=1}^{n} a_i=\sum_{i=1}^{n} b_i,$$

$$\sum_{i=1}^{k} a_i\leq\sum_{i=1}^{k}\min(b_i,k-1)+\sum_{i=k+1}^{n}\min(b_i,k).$$

  其中不等式須對所有 $1\leq k\leq n$ 成立。
- 測量師公式（鞋帶公式）
  設單純多邊形的頂點沿邊界依序為 $P_i=(x_i,y_i)$，$P_{n+1}=P_1$，則

$$S=\sum_{i=1}^{n}(x_i y_{i+1}-x_{i+1}y_i), \qquad A=\frac{|S|}{2}.$$

  可凹但不可自交；$S>0$ 為逆時針，$S<0$ 為順時針。整數座標可保留 $|S|=2A$；叉積與累加須防溢位。
- 邊界格點數
  沿用上述頂點順序，整數頂點的單純多邊形之邊界格點數為

$$B=\sum_{i=1}^{n}\gcd(|x_{i+1}-x_i|,|y_{i+1}-y_i|).$$

  單一線段含兩端點有 $\gcd(|\Delta x|,|\Delta y|)+1$ 個格點；多邊形各邊只加 gcd，避免重複計算頂點。搭配測量師公式與 Pick 定理可求內部格點數。
- Pick 定理
  對所有頂點為整數點的單純多邊形，有

$$A=\#\{\text{內部格點}\}+\frac{\#\{\text{邊界格點}\}}{2}-1.$$

- 莫比烏斯反演公式
  - $f(n)=\sum_{d|n} g(d) \Leftrightarrow g(n)=\sum_{d|n}\mu(d)f\left(\frac{n}{d}\right)$
  - $f(n)=\sum_{n|d} g(d) \Leftrightarrow g(n)=\sum_{n|d}\mu\left(\frac{d}{n}\right)f(d)$
- 整除分塊
  設整數 $n\geq 1$，從 $l=1$ 開始，每次計算

$$q=\left\lfloor\frac{n}{l}\right\rfloor, \qquad r=\left\lfloor\frac{n}{q}\right\rfloor.$$

  則 $i\in[l,r]$ 的 $\lfloor n/i\rfloor$ 都等於 q，且 r 為最右端。若 $r=n$ 即結束，否則令 $l=r+1$ 繼續，共 $O(\sqrt{n})$ 段。若 $A(t)=\sum_{i=1}^{t} a_i$ 可快速求得，則 $\sum_{i=l}^{r} a_i g(\lfloor n/i\rfloor)=[A(r)-A(l-1)]g(q)$；適合計算帶整除商的和式，乘法與累加須防溢位。
- 球冠（Spherical cap）
  - 球面被平面切下的一部分。
  - r: 球半徑，a: 球冠底面半徑，h: 球冠高度，$\theta=\arcsin(a/r)$。
  - 體積 $=\dfrac{\pi h^2(3r-h)}{3}=\dfrac{\pi h(3a^2+h^2)}{6}=\dfrac{\pi r^3(2+\cos\theta)(1-\cos\theta)^2}{3}$。
  - 表面積 $=2\pi rh=\pi(a^2+h^2)=2\pi r^2(1-\cos\theta)$。
- 拉格朗日乘數法
  - 在 k 個約束 $g_i(x_1,\ldots,x_n)=0$ 下優化 $f(x_1,\ldots,x_n)$。
  - 拉格朗日函數 $\mathcal{L}(x_1,\ldots,x_n,\lambda_1,\ldots,\lambda_k)=f(x_1,\ldots,x_n)-\sum_{i=1}^{k}\lambda_i g_i(x_1,\ldots,x_n)$。
  - 對應原始受限問題的解是拉格朗日函數的一個鞍點。

- 兩條錯線的最近點
  - Line 1 : $\boldsymbol{v}_1=\boldsymbol{p}_1+t_1\boldsymbol{d}_1$
  - Line 2 : $\boldsymbol{v}_2=\boldsymbol{p}_2+t_2\boldsymbol{d}_2$
  - $\boldsymbol{n}=\boldsymbol{d}_1\times\boldsymbol{d}_2$
  - $\boldsymbol{n}_1=\boldsymbol{d}_1\times\boldsymbol{n}$
  - $\boldsymbol{n}_2=\boldsymbol{d}_2\times\boldsymbol{n}$
  - $\boldsymbol{c}_1=\boldsymbol{p}_1+\dfrac{(\boldsymbol{p}_2-\boldsymbol{p}_1)\cdot\boldsymbol{n}_2}{\boldsymbol{d}_1\cdot\boldsymbol{n}_2}\boldsymbol{d}_1$
  - $\boldsymbol{c}_2=\boldsymbol{p}_2+\dfrac{(\boldsymbol{p}_1-\boldsymbol{p}_2)\cdot\boldsymbol{n}_1}{\boldsymbol{d}_2\cdot\boldsymbol{n}_1}\boldsymbol{d}_2$
- 導數／積分
  分部積分：$\int_a^b f(x)g(x)dx=[F(x)g(x)]_a^b-\int_a^b F(x)g'(x)dx$

$$\left|\begin{array}{ll}\dfrac{d}{dx}\sin^{-1}x=\dfrac{1}{\sqrt{1-x^2}} & \dfrac{d}{dx}\cos^{-1}x=-\dfrac{1}{\sqrt{1-x^2}} \\ \dfrac{d}{dx}\tan x=1+\tan^2 x & \displaystyle\int\tan(ax)dx=-\frac{\ln|\cos(ax)|}{a} \\ \displaystyle\int e^{-x^2}dx=\frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \displaystyle\int xe^{ax}dx=\frac{e^{ax}}{a^2}(ax-1)\end{array}\right| \quad \frac{d}{dx}\tan^{-1}x=\frac{1}{1+x^2}$$

$$\int\sqrt{a^2+x^2}dx=\frac{1}{2}\left(x\sqrt{a^2+x^2}+a^2\operatorname{asinh}(x/a)\right)$$

- 球座標（Spherical Coordinate）

$$(x,y,z)=(r\sin\theta\cos\phi,r\sin\theta\sin\phi,r\cos\theta)$$

$$(r,\theta,\phi)=\left(\sqrt{x^2+y^2+z^2},\arccos(z/\sqrt{x^2+y^2+z^2}),\operatorname{atan2}(y,x)\right)$$

- 旋轉矩陣

$$M(\theta)=\begin{bmatrix}\cos\theta & -\sin\theta\\ \sin\theta & \cos\theta\end{bmatrix},\quad R_x(\theta_x)=\begin{bmatrix}1 & 0 & 0\\ 0 & \cos\theta_x & -\sin\theta_x\\ 0 & \sin\theta_x & \cos\theta_x\end{bmatrix}$$

2.14 Estimation

整數分割數 $p(n)$：把 n 分成正整數和的不同方式數量。

n	2	3	4	5	6	7	8	9	20	30	40	50	100
$p(n)$	2	3	5	7	11	15	22	30	627	5604	4e4	2e5	2e8

最大因數數量 $\max_{i\le n}d(i)$：小於等於 n 的整數中，因數最多者的因數個數。

n	100	1e3	1e6	1e9	1e12	1e15	1e18
$d(i)$	12	32	240	1344	6720	26880	103680

中央二項式 $\binom{2n}{n}$：常用於卡塔蘭數與組合計數。

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\binom{2n}{n}$	2	6	20	70	252	924	3432	12870	48620	184756	7e5	2e6	1e7	4e7	1.5e8

貝爾數 B_n：將 n 個元素分成任意多個非空集合的劃分數。

n	2	3	4	5	6	7	8	9	10	11	12	13
B_n	2	5	15	52	203	877	4140	21147	115975	7e5	4e6	3e7

2.15 General Purpose Numbers

- 白努力數（Bernoulli numbers）
  $B_0=1, B_1^{\pm}=\pm\frac{1}{2}, B_2=\frac{1}{6}, B_3=0$
  滿足關係式 $\sum_{j=0}^{m}\binom{m+1}{j}B_j=0$。其指數生成函數（EGF）為

$$B(x)=\frac{x}{e^x-1}=\sum_{n=0}^{\infty}B_n\frac{x^n}{n!}.$$

  且

$$S_m(n)=\sum_{k=1}^{n}k^m=\frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k^+n^{m+1-k}.$$

- 第二類史特林數（Stirling numbers of the second kind）
  表示將 n 個不同元素分成恰好 k 個非空集合的劃分數。遞迴式：

$$S(n,k)=S(n-1,k-1)+kS(n-1,k),\quad S(n,1)=S(n,n)=1.$$

  顯式公式：

$$S(n,k)=\frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i}\binom{k}{i}i^n.$$

  轉換關係：

$$x^n=\sum_{i=0}^{n}S(n,i)(x)_i.$$

- 卡塔蘭數（Catalan numbers）

$$C_n^{(k)}=\frac{1}{(k-1)n+1}\binom{kn}{n},\qquad C^{(k)}(x)=1+x[C^{(k)}(x)]^k.$$

  其中 $C_n^{(2)}$ 為經典卡塔蘭數，常出現在二元樹、括號配對、Dyck path 等組合結構中。

- 歐拉數（Eulerian numbers）
  表示排列 $\pi\in S_n$ 中，恰有 k 個元素比前一個元素大的排列數。滿足遞迴式：

$$E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k),\quad E(n,0)=E(n,n-1)=1.$$

  顯式公式：

$$E(n,k)=\sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n.$$

3 Graph

3.1 DSU

```cpp
// 0-base；依大小合併 + 路徑壓縮，均攤 O(alpha(n))
// 代表點由合併順序決定，不保證是 a 或 b。
class dsu{
  public:
    vector<int> parent, siz;
    dsu(int num): parent(num), siz(num, 1){
      iota(parent.begin(), parent.end(), 0);
    }
    int find(int x){
      while(parent[x] != x){
        parent[x] = parent[parent[x]];
        x = parent[x];
      }
      return x;
    }
    bool same(int a, int b){
      return find(a) == find(b);
    }
    void Union(int a, int b){
      a = find(a), b = find(b);
      if(a == b) return;
      if(siz[a] < siz[b]) swap(a, b);
      parent[b] = a, siz[a] += siz[b];
    }
};
```

3.2 Dijkstra

```cpp
// 傳入圖的 pair 為 {權重, 點}, 無限大預設 1e9 是情況改
#define pii pair<int, int>
vector<
    int> dijkstra(vector<vector<pii>> &graph, int src){
  int n = graph.size();
  vector<int> dis(n, 1e9);
  vector<bool> vis(n, false);
  priority_queue<pii, vector<pii>, greater<pii>> pq;
  pq.push({0, src});
  dis[src] = 0;
  while(!pq.empty()){
    auto [w, node] = pq.top();
    pq.pop();
    if(vis[node]) continue;
    vis[node] = true;
    for(auto [nw, nn]:graph[node]){
      if(w + nw < dis[nn]){
        dis[nn] = w + nw;
        pq.push({dis[nn], nn});
      }
    }
  }
  return dis;
}
```

3.3 SPFA

```cpp
#define pii pair<int, int>
// {在 src 可到達
    的點中是否存在負環, 最短路徑}, arg 中 n 為點的數量
// arg 中 pair 裡的第一個值為權重, 第二個為點
pair<bool, vector<int>>
     SPFA(vector<vector<pii>> &graph, int n, int src){
  vector<int> dis(n + 1, 1e9);
  vector<int> cnt(n + 1, 0);
  vector<bool> vis(n + 1, false);
  queue<int> q;
  vis[src] = true; q.push(src); dis[src] = 0;
  while(!q.empty()){
    auto node = q.front(); vis[node] = false; q.pop();
    for(auto [w, nn]:graph[node]){
      if(w + dis[node] < dis[nn]){
        dis[nn] = w + dis[node];
        if(!vis[nn]){
```

```
          if(++cnt[nn] >= n) return {true, {}};
          q.push(nn);
          vis[nn] = true;
        }
      }
    }
  }
  return {false, dis};
}
```

3.4 Floyd Warshell

```
// 中繼點放外面

for (int k = 0; k < n; k++) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      dis[i
          ][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
    }
  }
}
```

3.5 Tarjan SCC

```
class tarjan{
  // 1-base；遞迴 DFS，深鏈須注意堆疊大小。
  int timer, id;
  stack<int> s;
  vector<int> low, dfn;
  vector<char> in_stack;
  void dfs(int u, vector<vector<int>> &graph){
    in_stack[u] = true, s.push(u);
    dfn[u] = low[u] = timer++;
    for(int v: graph[u]){
      if(!dfn[v]){
        dfs(v, graph);
        // 看看往下有沒有辦法回到更上面的點
        low[u] = min(low[u], low[v]);
      }
      else
          if(in_stack[v]) low[u] = min(low[u], dfn[v]);
    }
    if(low[u] != dfn[u]) return;
    vector<int> t; // 儲存這個強連通分量
    int v;
    do{
      v = s.top(), s.pop();
      in_stack[v] = false, scc_id[v] = id;
      t.push_back(v);
    }while(v != u);
    ++id, ans.push_back(t);
  }
  public:
    vector<int> scc_id;
    vector<vector<int>> ans;
    // ans ans[i] 代表第 i 個強連通分量裡面包涵的點
    // scc_id[i] 代表第 i 個點屬於第幾個強連通分量
    // 編號由 1 開始，較小者先被彈出；可重複呼叫。
    vector
        <vector<int>> scc(vector<vector<int>> &graph){
      int n = graph.size();
      timer = id = 1, s = {}, ans.clear();
      scc_id.assign(n, -1), dfn.assign(n, 0);
      low.assign(n, 0), in_stack.assign(n, false);
      for(int i = 1; i < n; i++)
        if(!dfn[i]) dfs(i, graph);
      return ans;
    }
};
```

3.6 2 SAT

```
// (a || b) && (c || d) && (e || f) .....

// 用
    下面的 tarjan scc 算法來解 2 sat 問題，若 事件 a 發
    生時，事件 b 必然發生，我們須在 a -> b 建立一條有向
// 用
     cses 的 Giant Pizza 來舉例子，給定 n 個人 m 個配料
    表，每個人可以提兩個要求，兩個要求至少要被滿足一個

// 3 5
// + 1 + 2
// - 1 + 3
// + 4 - 2

// 以這
    個例子來說，第一個人要求要加 配料1 或者 配料2 其中
    一項，第二個人要求不要 配料1 或者 要配料3 其中一項
// 試問能不能滿足所有人的要求，我們可以把 要加
    配料 i 當作點 i ，不加配料 i 當作點 i + m(配料數量)
// 關於第一個人的要求 我們可以看成若不加 配
    料1 則必定要 配料2 以及 若不加 配料2 則必定要 配料1
// 關於第二個人要求 可看做加了 配料
    1 就必定要加 配料3 以及 不加 配料3 就必定不加 配料1

// 以這些條件建立有向圖，並且
    找尋 scc ，若 i 以及 i + m 在同一個 scc 中代表無解
// 若要求解，則若 i 的 scc_id
     小於 i + m 的 scc_id 則 i 為 true ，反之為 false
// tarjan 的模板在上面
cin >> n >> m;

vector<vector<int>> graph(m * 2 + 1);
function<int(int)> tr = [&](int x){
  if(x > m) return x - m;
  return x + m;
};

for(int i = 0; i < n; i++){
  char c1, c2;
  int a, b;
  cin >> c1 >> a >> c2 >> b;
  // a 代表 a 為真，m + a 代表 a 為假
  if(c1 == '-') a += m;
  if(c2 == '-') b += m;
  graph[tr(a)].push_back(b);
  graph[tr(b)].push_back(a);
}

tarjan t;
auto scc = t.scc(graph);

for(int i = 1; i <= m; i++){
  if(t.scc_id[i] == t.scc_id[tr(i)]){
    cout << "IMPOSSIBLE\n";
    return 0;
  }
}

for(int i = 1; i <= m; i++){
  if(t.scc_id[i] < t.scc_id[tr(i)]){
    cout << '+';
  }
  else cout << '-';
  cout << ' ';
}
cout << '\n';
```

3.7 Euler Path

```
// 1. 無向圖是歐拉圖：
// 非零度頂點是連通的
// 頂點的度數都是偶數

// 2. 無向圖是半歐拉圖(有路沒有環)：
// 非零度頂點是連通的
// 恰有 2 個奇度頂點

// 3. 有向圖是歐拉圖：
// 非零度頂點是強連通的
// 每個頂點的入度和出度相等

// 4. 有向圖是半歐拉圖(有路沒有環)：
// 非零度頂點是弱連通的
// 至多一個頂點的出度與入度之差為 1
// 至多一個頂點的入度與出度之差為 1
// 其他頂點的入度和出度相等
vector<set<int>> adj;
vector<int> ans;

void dfs(int x) { // Hierholzer's Algorithm
  while (!adj[x].empty()) {
    auto next = *(adj[x].begin());
    adj[x].erase(next);
```

```
    adj[next].erase(x);
    dfs(next);
  }
  ans.emplace_back(x);
}

void solve() {
  // 建立雙向邊，set用來防重邊，點數n，邊數m
  for (int i = 1; i <= n; i++)
    if (adj[i].size() & 1) return; /* impossible */
  dfs(1);
  if (ans.size() != m + 1) return; /* impossible */
  reverse(ans.begin(), ans.end()); /* then print it */
}
```

3.8 Bridge

```
// 橋: 若移除邊會使連通分量變多
// [USAGE] ECC ecc(n); ecc.add_edge(u, v); ecc.solve();
// is_bridge[i]; necc; bln[i];
// 邊是否為橋; 橋連通分量數量; 頂點所屬橋連通分量編號
struct ECC { // 0-base
  int n, dft, ecnt, necc;
  vector<int> low, dfn, bln, is_bridge, stk;
  vector<vector<pii>> G;
  void dfs(int u, int f) {
    dfn[u] = low[u] = ++dft, stk.pb(u);
    for (auto [v, e] : G[u])
      if (!dfn[v])
        dfs(v, e), low[u] = min(low[u], low[v]);
      else if (e != f)
        low[u] = min(low[u], dfn[v]);
    if (low[u] == dfn[u]) {
      if (f != -1) is_bridge[f] = 1;
      for (; stk.back() != u; stk.pop_back())
        bln[stk.back()] = necc;
      bln[u] = necc++, stk.pop_back();
    }
  }
  ECC(int _n): n(_n), dft()
      , ecnt(), necc(), low(n), dfn(n), bln(n), G(n) {}
  void add_edge(int u, int v) {
    G[u].pb(pii(v, ecnt)), G[v].pb(pii(u, ecnt++));
  }
  void solve() {
    is_bridge.resize(ecnt);
    for (int i = 0; i < n; ++i)
      if (!dfn[i]) dfs(i, -1);
  }
};  // 8BQube
```

3.9 Max flow min cut

```
#define int long long

// dicnic Algorithm Time: O(V^2E) 實際上會快一點
// 記得在 main 裡面 resize graph
// 最小割，找
    到最少條的邊切除，使得從 src 到 end 的 maxflow 為 0
// 枚舉所有邊 i -> j，src 可
    以到達 i 但無法到達 j ，那這條邊為最小割裡的邊之一
// 無向圖最大流：修改 add_edge，反向邊建為 capacity
// 使用時只要 add_edge 一次
// 「最少條」限等正容量；一般為容量總和最小。
// 多測資
    用 graph.assign(n, {})；容量非負，總流量不可溢位。
// maxflow 回傳本次新增流量；start == end 回傳 0。
struct edge{
  int next, origin, capacity, rev;
  bool is_rev;
  edge(int n, int c, int r, bool ir)
    : next(n)
      , origin(c), capacity(c), rev(r), is_rev(ir) {}
};
vector<vector<edge>> graph;
vector<int> level, iter;

void add_edge(int a, int b, int capacity){
  int i = graph[a].size(), j = graph[b].size();
  graph[a
      ].emplace_back(b, capacity, j + (a == b), false);
  graph[b].emplace_back(a, 0, i, true);
}

void bfs(int start){
  fill(level.begin(), level.end(), -1);
  queue<int> q;
  level[start] = 0, q.push(start);
  while(!q.empty()){
    int v = q.front(); q.pop();
    for(auto &e: graph[v])
      if(e.capacity > 0 && level[e.next] < 0)
        level[e.next] = level[v] + 1, q.push(e.next);
  }
}

int dfs(int v, int end, int flow){
  if(v == end) return flow;
  for(int &i = iter[v]; i < (int)graph[v].size(); i++){
    auto &e = graph[v][i];
    if(e.
        capacity <= 0 || level[e.next] != level[v] + 1)
      continue;
    int d = dfs(e.next, end, min(flow, e.capacity));
    if(!d) continue;
    e.capacity
        -= d, graph[e.next][e.rev].capacity += d;
    return d;
  }
  return 0;
}

int maxflow(int start, int end){
  if(start == end) return 0;
  int flow = 0, f;
  level.resize(graph.size());
  while(true){
    bfs(start);
    if(level[end] < 0) return flow;
    iter.assign(graph.size(), 0);
    while((f = dfs(start, end, LLONG_MAX))) flow += f;
  }
}

// 回朔 max_flow 的 path
// 用法：graph.assign(n, {}); add_edge(...);
// int f
    = maxflow(s, t); auto paths = get_flow_paths(s, t);
// 每項 {path, flow}；會消耗同一組 s,t 已送的流量。
// 限上述有向邊模型的 maxflow 結果，不任意更換源匯。
// 分解
    最壞 O(VE)，保留不連通環流；Dinic DFS 有深度限制。
struct flow_path{ vector<int> path; int flow; };
vector<flow_path> get_flow_paths(int start, int end){
  vector<flow_path> result;
  if(start == end) return result;
  vector<int> ptr(graph.size()), pos(graph.size(), -1);
  vector<int> path{start}, edges;
  pos[start] = 0;
  // 原始邊目前剩下多少尚未分解的流量
  auto used = [&](int u, int i){
    return graph[u][i].origin - graph[u][i].capacity;
  };
  // 保留 path[0..last]
  auto trim = [&](int last){
    while((int)path.size() > last + 1){
      pos[path.back()] = -1; path.pop_back();
    }
    edges.resize(last);
  };
  while(true){
    int v = path.back(), k = 0;
    if(v != end){
      int &i = ptr[v];
      while(i < (int)graph[v].size() &&
            (graph
                [v][i].is_rev || used(v, i) <= 0)) i++;
      // 流量守恆：走進中間點後，一定有正流量出邊
      if(i == (int)graph[v].size()){
        assert(v == start); break;
      }
      int u = graph[v][i].next;
      edges.push_back(i);
      if(pos[u] == -1){
        pos[u] = path.size(); path.push_back(u);
        continue;
      }
      // 發現環：path[k] -> ... -> v -> path[k]
```

```
      k = pos[u]; // edges 已含最後一條回到入口的邊
      // 消去環流，不影響 s -> t 的總流量
    }
    // 路徑取全部邊，環只取 k 起的邊；共用扣流流程。
    int f = LLONG_MAX, cut = edges.size();
    for(int j = k; j < (int)edges.size(); j++)
      f = min(f, used(path[j], edges[j]));
    if(v == end) result.push_back({path, f});
    // 找到第一條流量歸零的邊
    for(int j = k; j < (int)edges.size(); j++){
      auto &e = graph[path[j]][edges[j]];
      // 消耗這條邊的 f 單位流量
      e.capacity
           += f, graph[e.next][e.rev].capacity -= f;
      if(!used(path[j], edges[j])) cut = min(cut, j);
    }
    // 保留仍有流量的前綴，不必從起點重走
    trim(v == end ? cut : k);
  }
  return result;
}
```

3.10 Minimum cost maximum flow

```
#define int long long
#define pii pair<int, int>

// 逐次最短增廣路；K 次增廣，最壞 O(KVE)
// 一條邊的費用為 單位花費 * 流過流量
// 把原本的 BFS 換成 SPFA 而已
// 記得在 main 裡面 resize graph；多測資用 assign 清空
// MCMF 回傳 {flow, cost}；會修改殘餘容量
// 無向圖：add_edge(u,v,C,W), add_edge(v,u,C,W);
// 容量非負；不可有源點可達的負環。
// 距離絕對值 < INF；容量、總費用不可溢位。
const int INF = LLONG_MAX / 4;

struct edge{
  int next, capacity, rev, cost;
  bool is_rev;
};
vector<vector<edge>> graph;

void add_edge(int a, int b, int capacity, int cost){
  int i = graph[a].size(), j = graph[b].size();
  graph[a].push_back({b, capacity, j + (a == b),
                      cost, false});
  graph[b].push_back({a, 0, i, -cost, true});
}

pii MCMF(int start, int end){
  if(start == end) return {0, 0};
  int n = graph.size(), flow = 0, cost = 0;
  vector<int> dis(n), parent(n), pe(n);
  vector<char> vis(n);
  while(true){
    fill(dis.begin(), dis.end(), INF);
    fill(vis.begin(), vis.end(), false);
    queue<int> q;
    dis[start] = 0, vis[start] = true, q.push(start);
    while(!q.empty()){
      int u = q.front(); q.pop(); vis[u] = false;
      for(int i = 0; i < (int)graph[u].size(); i++){
        auto &e = graph[u][i];
        if(e.capacity <= 0 ||
           dis[e.next] <= dis[u] + e.cost)
          continue;
        dis[e.next] = dis[u] + e.cost;
        parent[e.next] = u, pe[e.next] = i;
        if(!vis[e.next])
          vis[e.next] = true, q.push(e.next);
      }
    }
    if(dis[end] == INF) return {flow, cost};
    int add = LLONG_MAX;
    for(int v = end; v != start; v = parent[v])
      add = min(add, graph[parent[v]][pe[v]].capacity);
    for(int v = end; v != start; v = parent[v]){
      auto &e = graph[parent[v]][pe[v]];
      e.capacity -= add;
      graph[v][e.rev].capacity += add;
    }
    flow += add, cost += add * dis[end];
  }
}
```

3.11 二分圖

```
/*
判定二分圖：著色法 dfs 下去，顏色相撞非二分圖

二分圖最大匹配：用 maxflow 去做，一個 src
    點聯通所有左圖，左圖建邊向右圖，右圖再建邊向 end
    點，計算 src 跟 end 的最大流，若要還原，找出左圖
    通往右圖中 capacity 為 0 的邊，他的兩個端點就是答案

最小點覆蓋：選最少的點，保證每條邊
    至少有一個端點被選到， 最小點覆蓋 = 二分圖最大匹配

最大獨立集：選最多的點，滿足這些
    點兩兩間互不相連， 最大獨立集 = n - 二分圖最大匹配

還原解，flow 的作
    法是從源點開始 dfs，只走 cap - flow > 0 的邊，最後
    挑選左邊還沒被跑過的點和右邊被跑過的點當作覆蓋的點
*/
```

3.12 KM 匈牙利

```
using ll = long long;
// KM（Kuhn-Munkres / Hungarian）
//   最小權完美匹配，O(n^2 m)
// 來源：cp-algorithms Hungarian
//    （Andrey Lopatin，O(n^3) 版）
// a：n x m 權重矩陣，0-based，需 n <= m（否則 assert）
// ans[i]：第 i 列配到的行（0-based）；回傳最小總權
// 求最大權：a 全取負呼叫，結果取負回去；負權安全。
ll km(const vector<vector<ll>>& a, vector<int>& ans) {
  int n = (int)a.size();
  ans.assign(n, -1);
  if (n == 0) return 0;
  int m = (int)a[0].size();
  assert(n <= m);
  const ll INF = (ll)4e18;
  vector<ll> u(n + 1), v(m + 1);
  // p[j] 1-based，0 = 未配
  vector<int> p(m + 1), way(m + 1);
  for (int i = 1; i <= n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<ll> minv(m + 1, INF);
    vector<char> used(m + 1);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      ll delta = INF;
      for (int j = 1; j <= m; ++j) if (!used[j]) {
        ll cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < minv[j])
          minv[j] = cur, way[j] = j0;
        if (minv[j] < delta) delta = minv[j], j1 = j;
      }
      for (int j = 0; j <= m; ++j)
        if (used[j]) u[p[j]] += delta, v[j] -= delta;
        else minv[j] -= delta;
      j0 = j1;
    } while (p[j0] != 0);
    do {
      int j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0 != 0);
  }
  for (int j = 1; j <= m; ++j)
    if (p[j] != 0) ans[p[j] - 1] = j - 1;
  return -v[0];
}

// 無權二分圖轉 KM 矩陣，O(V + E + L R)
// graph：0-based 鄰接表，單向或雙向存邊皆可
// color：已染好 0 / 1，邊須異色
// L、R 為兩側點數；小側當列，同大時用 color 0
// 列、行各依原頂點編號遞增；回傳 ll 矩陣
// 有邊 -1，沒邊 0（補位）；-km 即最大匹配數
// ans 為矩陣編號，a[i][ans[i]] == -1 才是原邊
// 用法：auto a = km_matrix(graph, color);
```

```
//       vector<int> ans;
//       ll match = -km(a, ans);
// 最大匹配 = 最小點覆蓋；最大獨立集 = V - match
vector<vector<ll>> km_matrix(
    const vector<vector<int>>& graph,
    const vector<int>& color) {
  int n = (int)graph.size(), cnt[2] = {};
  vector<int> id(n);
  for (int i = 0; i < n; ++i) {
    id[i] = cnt[color[i]]++;
  }
  int side = cnt[0] > cnt[1];
  vector<vector<ll>> a(cnt[side],
      vector<ll>(cnt[side ^ 1]));
  for (int u = 0; u < n; ++u) for (int v : graph[u]) {
    if (color[u] == side) a[id[u]][id[v]] = -1;
    else a[id[v]][id[u]] = -1;
  }
  return a;
}
```

3.13 Check cycle

```
vector<int> G[MAXN];
bool visit[MAXN];
/* return if the connected component where u is
    contains a cycle*/
bool dfs(int u, int pre) {
    if(visit[u])    return true;
    visit[u] = true;
    for(int v : G[u])
        if(v != pre && dfs(v, u))
            return true;
    return false;
}

//check if a graph contains a cycle

bool checkCycle(int n) {
    for(int i = 1; i <= n; i++)
        if(!visit[i] && dfs(i, -1))
            return true;
    return false;
}
```

3.14 BCC

```
// [USAGE] BCC bcc(n); bcc.add_edge(u, v); bcc.solve();
// bcc.is_ap[i]; // i 是否為割點
// bcc.bcc[j]; // 第 j 個點雙連通分量中包含的所有頂點
// bcc.nbcc; // 點雙連通分量數量

// [USAGE] bcc.block_cut_tree();
// bcc.nG[i]; // 新圖頂點 i 的所有鄰居
// bcc
    .bln[i]; // 原圖中的 i 在新圖上的編號(i可以是割點)
// bcc.cir[j]; // 新圖上的頂點 j 是否為割點
struct BCC { // 0-base
  int n, dft, nbcc;
  vector<int> low, dfn, bln, stk, is_ap, cir;
  vector<vector<int>> G, bcc, nG;
  void make_bcc(int u) {
    bcc.emplace_back(1, u);
    for (; stk.back() != u; stk.pop_back())
      bln[stk.back()] = nbcc, bcc[nbcc].pb(stk.back());
    stk.pop_back(), bln[u] = nbcc++;
  }
  void dfs(int u, int f) {
    int child = 0;
    low[u] = dfn[u] = ++dft, stk.pb(u);
    for (int v : G[u])
      if (!dfn[v]) {
        dfs(v, u), ++child;
        low[u] = min(low[u], low[v]);
        if (dfn[u] <= low[v]) {
          is_ap[u] = 1, bln[u] = nbcc;
          make_bcc(v), bcc.back().pb(u);
        }
      } else if (dfn[v] < dfn[u] && v != f)
        low[u] = min(low[u], dfn[v]);
    if (f == -1 && child < 2) is_ap[u] = 0;
    if (f == -1 && child == 0) make_bcc(u);
  }
  BCC(int _n): n(_n), dft(),
      nbcc(), low(n), dfn(n), bln(n), is_ap(n), G(n) {}
  void add_edge(int u, int v) {
    G[u].pb(v), G[v].pb(u);
  }
  void solve() {
    for (int i = 0; i < n; ++i)
      if (!dfn[i]) dfs(i, -1);
  }
  void block_cut_tree() {
    cir.resize(nbcc);
    for (int i = 0; i < n; ++i)
      if (is_ap[i])
        bln[i] = nbcc++;
    cir.resize(nbcc, 1), nG.resize(nbcc);
    for (int i = 0; i < nbcc && !cir[i]; ++i)
      for (int j : bcc[i])
        if (is_ap[j])
          nG[i].pb(bln[j]), nG[bln[j]].pb(i);
  } // up to 2 * n - 2 nodes!! bln[i] for id
};  // 8BQube
```

4 String

4.1 trie

```
// 僅接受 a..z；擁有整棵樹，不可複製
// 解構採遞迴，極深單字需改用節點池
class trie {
 public:
  struct node {
    int count = 0;
    vector<node*> child = vector<node*>(26, nullptr);
    ~node() {
      for (auto c : child) delete c;
    }
  };
  node* root = new node;
  trie() = default;
  trie(const trie&) = delete;
  trie& operator=(const trie&) = delete;
  ~trie() { delete root; }
  void insert(const string& s) {
    auto p = root;
    for (char c : s) {
      int k = c - 'a';
      if (!p->child[k]) p->child[k] = new node;
      p = p->child[k];
    }
    ++p->count;
  }
  bool search(const string& s) const {
    auto p = root;
    for (char c : s) {
      p = p->child[c - 'a'];
      if (!p) return false;
    }
    return p->count > 0;
  }
};
```

4.2 KMP

```
// 計算可重疊出現次數；空模式匹配 n + 1 個位置
vector<int> build(const string &s){
  vector<int> next(s.empty() ? 1 : 2);
  // 匹配失敗跳去哪 (最長共同前後綴)
  int length = s.size(), j = 0;
  for(int i = 1; i < length; i++){
    while(j > 0 and s[j] != s[i]){
      j = next[j];
    }
    if(s[j] == s[i]) j++;
    next.push_back(j);
  }
  return next;
}

int match(const string &a, const string &b){
  if (b.empty()) return (int)a.size() + 1;
  auto next = build(b);
  int length = a.size(), length2 = b.size();
  int j = 0, count = 0;
  for(int i = 0; i < length; i++){
    while(j > 0 and a[i] != b[j]){
      j = next[j];
    }
    if(a[i] == b[j]) j++;
```

```
    if(j == length2){
      count++;
      j = next[j];
    }
  }
  return count;
}
```

4.3 Hash

```
vector<int> Pow(int num){
  int p = 1e9 + 7;
  vector<int> ans = {1};
  for(int i = 0; i < num; i++)
    ans.push_back(ans.back() * b % p);
  return ans;
}

vector<int> Hash(string s){
  int p = 1e9 + 7;
  vector<int> ans = {0};
  for(char c:s){
    ans.push_back((ans.back() * b + c) % p);
  }
  return ans;
}

// 閉區間[l, r]
int query
    (vector<int> &vec, vector<int> &pow, int l, int r){
  int p = 1e9 + 7;
  int length = r - l + 1;
  return
       (vec[r + 1] - vec[l] * pow[length] % p + p) % p;
}
```

4.4 Zvalue

```
vector<int> z_func(string s1){
  int l = 0, r = 0, n = s1.size();
  vector<int> z(n, 0);
  for(int i = 1; i < n; i++){
    if(i
        <= r and z[i - l] < r - i + 1) z[i] = z[i - l];
    else{
      z[i] = max(z[i], r - i + 1);
      while(i + z
          [i] < n and s1[i + z[i]] == s1[z[i]]) z[i]++;
    }
    if(i + z[i] - 1 > r){
      l = i;
      r = i + z[i] - 1;
    }
  }
  return z;
}
```

4.5 最長迴文子字串

```
// 找到對於每個位置的迴文半徑
vector<int> manacher(string s) {
  string t = "#";
  for (auto c : s) {
    t += c;
    t += '#';
  }
  int n = t.size();
  vector<int> r(n);
  for (int i = 0, j = 0; i
       < n; i++) {    // i 是中心, j 是最長回文字串中心
    if (2 * j - i >= 0 && j + r[j] > i) {
      r[i] = min(r[2 * j - i], j + r[j] - i);
    }
    while (i - r[i] >= 0 &&
          i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
      r[i] += 1;
    }
    if (i + r[i] > j + r[j]) {
      j = i;
    }
  }
  return r;
  // # a # b # a #
  // 1 2 1 4 1 2 1
  // # a # b # b # a #
  // 1 2 1 2 5 2 1 2 1
  // 值 -1 代表原回文字串長度
  // (id - val + 1) / 2 可得原字串回文開頭
}
```

4.6 Suffix Array

```
// 空字串回傳空陣列；O(n log n) 時間、O(n) 空間
struct SuffixArray {
  int n; string s;
  vector<int> sa, rk, lc;
  // 想法 :
       排序過了，因此前綴長得像的會距離很近在差不多位置
  // n: 字串長度
  // sa: 後綴數組，sa[i] 表示第 i 小的後綴的起始位置
  // rk: 排名數組，rk[i] 表示從位置 i 開始的後綴的排名
  // lc: LCP 數組，
       lc[i] 表示 sa[i] 和 sa[i + 1] 的最長公共前綴長度
  // 求 sa[i] 跟 sa[j] 的
       LCP 長度 當 i < j : min(lc[i] ..... lc[j - 1])
  // 求 longest common substring : A +
       "#" + B 建立 SA，找到 sa 相鄰但不同組中 lc 最大的
  SuffixArray(const string &s_) {
    s = s_; n = s.length();
    sa.resize(n);
    if (!n) return;
    lc.resize(n - 1);
    rk.resize(n);
    iota(sa.begin(), sa.end(), 0);
    sort(sa.begin(), sa.end(), [&](int a, int b) {
      return (unsigned char)s[a] < (unsigned char)s[b];
    });
    rk[sa[0]] = 0;
    for (int i = 1; i < n; ++i)
      rk[sa[i]]
          = rk[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    int k = 1;
    vector<int> tmp, cnt(n);
    tmp.reserve(n);
    while (rk[sa[n - 1]] < n - 1) {
      tmp.clear();
      for (int i = 0; i < k; ++i)
        tmp.push_back(n - k + i);
      for (auto i : sa)
        if (i >= k)
          tmp.push_back(i - k);
      fill(cnt.begin(), cnt.end(), 0);
      for (int i = 0; i < n; ++i)
        ++cnt[rk[i]];
      for (int i = 1; i < n; ++i)
        cnt[i] += cnt[i - 1];
      for (int i = n - 1; i >= 0; --i)
        sa[--cnt[rk[tmp[i]]]] = tmp[i];
      swap(rk, tmp);
      rk[sa[0]] = 0;
      auto second = [&](int i) {
        return i + k < n ? tmp[i + k] : -1;
      };
      for (int i = 1; i < n; ++i) {
        int a = sa[i - 1], b = sa[i];
        rk[b] = rk[a] + (tmp[a] != tmp[b] ||
                         second(a) != second(b));
      }
      k *= 2;
    }
    for (int i = 0, j = 0; i < n; ++i) {
      if (rk[i] == 0) {
        j = 0;
      } else {
        int prev = sa[rk[i] - 1];
        for (j -= j > 0; i + j < n && prev + j < n
             && s[i + j] == s[prev + j]; )
          ++j;
        lc[rk[i] - 1] = j;
      }
    }
  }
};
```

4.7 AC-Automatan

```
// A 起算的 sigma 字元；len 須容納根與所有節點
struct AC_Automatan {
  int nx[len][sigma], fl[len], cnt[len], ord[len], top;
  int rnx[len][sigma]; // node actually be reached
```

```
  int newnode() {
    fill_n(nx[top], sigma, -1);
    cnt[top] = 0;
    return top++;
  }
  // 每輪先 init，再插字串、make_fl、掃描並 solve 一次
  void init() { top = 1, newnode(); }
  int input(string &s) {
    int X = 1;
    for (char c : s) {
      if (!~nx[X][c - 'A']) nx[X][c - 'A'] = newnode();
      X = nx[X][c - 'A'];
    }
    return X; // return the end node of string
  }
  void make_fl() {
    queue<int> q;
    q.push(1), fl[1] = 0;
    for (int t = 0; !q.empty(); ) {
      int R = q.front();
      q.pop(), ord[t++] = R;
      for (int i = 0; i < sigma; ++i)
        if (~nx[R][i]) {
          int X = rnx[R][i] = nx[R][i];
          fl[X] = R == 1 ? 1 : rnx[fl[R]][i];
          q.push(X);
        }
        else rnx[R][i] = R > 1 ? rnx[fl[R]][i] : 1;
    }
  }
  void solve() {
    for (int i = top - 2; i > 0; --i)
      cnt[fl[ord[i]]] += cnt[ord[i]];
  }
} ac;
```

4.8 Duval

```
// duval_algorithm
// 將字串分解成若干個非嚴格遞減的非嚴格遞增字串
vector<string> duval(string s) {
    int i = 0, n = s.size();
    vector<string> res;
    while (i < n) {
        int k = i, j = i + 1;
        while (s[k] <= s[j] && j < n) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) {
            res.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return res;
}
// 最小
    旋轉字串 一個字串所有循環同構（即將字串頭尾相連後，
    從不同位置開始取字串）的字串中，字典序最小的那一個
string minRound(string s) {
    s += s;
    int i = 0, n = s.size(), start = i;
    while (i < n / 2) {
        start = i;
        int k = i, j = i + 1;
        while (s[k] <= s[j] && j < n) {
            if (s[k] < s[j]) k = i;
            else k++;
            j++;
        }
        while (i <= k) i += j - k;
    }
    return s.substr(start, n / 2);
}
```

5 Geometry

5.1 Point

```
template<typename T>
struct point {
  T x{}, y{};
  point() = default;
  point(T x, T y) : x(x), y(y) {}
  point operator+(const point& a) const {
    return {x + a.x, y + a.y};
  }
  point operator-(const point& a) const {
    return {x - a.x, y - a.y};
  }
  point operator/(const point& a) const {
    return {x / a.x, y / a.y};
  }
  point operator/(T a) const { return {x / a, y / a}; }
  point operator*(T a) const { return {x * a, y * a}; }
  bool operator<(const point& a) const {
    return x != a.x ? x < a.x : y < a.y;
  }
  bool operator==(const point& a) const {
    return x == a.x && y == a.y;
  }
};
```

5.2 內積, 外積, 距離

```
template<typename T>
T dot(const point<T> &a,const point<T> &b){
    return a.x * b.x + a.y * b.y;
}

template<typename T>
T cross(const point<T> &a,const point<T> &b){
    return a.x * b.y - a.y * b.x;
}

template<typename T>
T len(point<T> p){
    return sqrt(dot(p, p));
}

template<typename T>
int sign(T x){
    return x == 0 ? 0 : x > 0 ? 1 : -1;;
}

template<typename T>
T pointSegDist(point<T> q0, point<T> q1, point<T> p) {
    if (sign(len(q0 - q1)) == 0) return len(q0 - p);
    if (sign(dot(q1 - q0, p
        - q0)) >= 0 && sign(dot(q0 - q1, p - q1)) >= 0)
        return
            abs(cross(q1 - q0, p - q0) / len(q0 - q1));
    return min(len(p - q0), len(p - q1));
}
```

5.3 向量應用

```
// cross、dot 的中間結果須不溢位；浮點判斷採精確比較
template<typename T>
bool collinearity(point<T> a, point<T> b, point<T> c) {
  //檢查三點是否共線
  return cross(b - a, c - a) == 0;
}

template<typename T>
bool inLine(point<T> a, point<T> b, point<T> p) {
  //檢查 p 點是否在ab線段
  return
      collinearity(a, b, p) && dot(a - p, b - p) <= 0;
}

template<typename T>
bool intersect(point<T> a, point<T> b,
               point<T> c, point<T> d) {
  //ab線段跟cd線段是否相交
  auto opposite = [](T x, T y) {
    return (x < 0 && y > 0) || (x > 0 && y < 0);
  };
  return (opposite(cross(b - a, c - a),
                   cross(b - a, d - a)) &&
          opposite(cross(d - c, a - c),
                   cross(d - c, b - c))) ||
         inLine(a, b, c) || inLine(a, b, d) ||
         inLine(c, d, a) || inLine(c, d, b);
}

template<typename T>
point<T> intersection(point<T> a, point<T> b,
                      point<T> c, point<T> d) {
  //ab線段跟cd線段相交的點
```

```
  // T 用浮點數；要求唯一交點，不接受共線重疊
  assert(intersect(a, b, c, d));
  T den = cross(d - c, b - a);
  assert(den != 0);
  return a + (b - a) * (cross(a - c, d - c) / den);
}

template<typename T>
bool inPolygon(const vector<point<T>>& polygon,
               point<T> p) {
  //判斷
      點p是否在多邊形polygon裡，vector裡的點要連續填對
  // 簡單多邊形，含邊界、支援凹形與兩種方向
  bool inside = false;
  int n = polygon.size();
  for (int i = 0; i < n; ++i) {
    auto a = polygon[i], b = polygon[(i + 1) % n];
    if (inLine(a, b, p)) return true;
    if ((a.y > p.y) == (b.y > p.y)) continue;
    auto turn = cross(b - a, p - a);
    if ((turn > 0) == (b.y > a.y)) inside = !inside;
  }
  return inside;
}

template<typename T>
long double triangleArea(point<T> a, point<T> b,
                         point<T> c) {
  //三角形頂點，求面積
  return fabs((long double)cross(b - a, c - a)) / 2;
}

template<typename T, typename F, typename S>
long
    double triangleArea_Herons_formula(T a, F b, S c) {
  //三角形頂點，求面積(給邊長)
  long double x = a, y = b, z = c;
  long double p = (x + y + z) / 2;
  assert(x >= 0 && y >= 0 && z >= 0 &&
         x + y >= z && y + z >= x && z + x >= y);
  return
      sqrtl(max(0.L, p * (p - x) * (p - y) * (p - z)));
}

template<typename T>
long double area(const vector<point<T>>& p) {
  //多邊形頂點，求面積
  // 以首點為原點，減少大座標平移造成的浮點抵消
  long double sum = 0;
  for (int i = 1; i + 1 < (int)p.size(); ++i)
    sum += (long double)cross(p[i] - p[0],
                              p[i + 1] - p[0]);
  return fabs(sum) / 2;
}
```

5.4 Static Convex Hull

```
// 需要使
    用前一個向量模板的 point ， 需要 operator - 以及 <
// 需要前面向量模板的 cross
// 回傳逆時針凸包，保留邊界點、不重複首點；會排序輸入
// 全共線時回傳排序去重的點；cross 須不溢位
template<typename T>
vector<point<T>> getConvexHull(vector<point<T>>& pnts){
  sort(pnts.begin(), pnts.end());
  auto same = [](point<T> a, point<T> b) {
    return a.x == b.x && a.y == b.y;
  };
  pnts.erase(unique(pnts.begin(), pnts.end(), same),
             pnts.end());
  int n = pnts.size();
  if (n <= 2) return pnts;
  bool flat = true;
  for (auto p : pnts)
    flat &=
        cross(pnts.back() - pnts[0], p - pnts[0]) == 0;
  if (flat) return pnts;
  vector<point<T>> hull;
  for (int side = 0; side < 2; ++side) {
    int start = hull.size();
    for (auto p : pnts) {
      while ((int)hull.size() - start >= 2) {
        auto
            a = hull[hull.size() - 2], b = hull.back();
        // <= 0 或者 < 0 要看點有沒有在邊上
        if (cross(b - a, p - a) >= 0) break;
        hull.pop_back();
      }
      hull.push_back(p);
    }
    hull.pop_back();
    reverse(pnts.begin(), pnts.end());
  }
  return hull;
}
```

5.5 外心, 最小覆蓋圓

```
int sign(double a) {
  // 小於 eps
       回傳 0，否則正回傳 1 ，負回傳 應付浮點數誤差用
  const double eps = 1e-10;
  return fabs(a) < eps ? 0 : a > 0 ? 1 : -1;
}

// 輸入三個點求外心
// T 用浮點數；三點不可共線，eps 為叉積容許值
template<typename T>
point<T> findCircumcenter(point<T> A, point<T> B,
                          point<T> C, T eps = 1e-10) {
  auto AB = B - A, AC = C - A;
  T ab = AB.x * AB.x + AB.y * AB.y;
  T ac = AC.x * AC.x + AC.y * AC.y;
  T d = AB.x * AC.y - AB.y * AC.x;
  // 若三點接近共線
  assert(fabs(d) >= eps && d != 0);
  // 外心的座標
  return A + point<T>(AC.y * ab - AB.y * ac,
                      AB.x * ac - AC.x * ab) / (2 * d);
}

template<typename T>
pair<T, point<T>> MinCircleCover(vector<point<T>>& p) {
  // 引入前面的 len 跟 point
  // 回傳最小覆蓋圓{半徑,中心}
  // 會打亂輸入；空集合回傳 {0, (0, 0)}；期望 O(n)
  // 直接比較容差，避免與其他模板的 sign 多載衝突
  const T eps = 1e-10;
  if (p.empty()) return {0, {0, 0}};
  static mt19937 rng(712367);
  shuffle(p.begin(), p.end(), rng);
  int n = p.size();
  point<T> c = p[0];
  T r = 0;
  for (int i = 1; i < n; ++i) {
    if (len(c - p[i]) - r < eps) continue;
    // 不在圓內
    c = p[i], r = 0;
    for (int j = 0; j < i; ++j) {
      if (len(c - p[j]) - r < eps) continue;
      c = (p[i] + p[j]) / T(2);
      r = len(c - p[i]);
      for (int k = 0; k < j; ++k) {
        if (len(c - p[k]) - r < eps) continue;
        c = findCircumcenter(p[i], p[j], p[k], T(0));
        r = len(c - p[i]);
      }
    }
  }
  return {r, c};
}
```

5.6 最小矩形覆蓋

```
// 向量長度平方
template<typename T>
T square(const point<T>& a) {
  return a.x * a.x + a.y * a.y;
}

// 向量逆時針旋轉 90 度
template<typename T>
point<T> rot(const point<T>& a) { return {-a.y, a.x}; }

// 直線定義：通過兩點 a, b
template<typename T>
struct Line { point<T> a, b; };
```

```cpp
// 兩直線交點 (假設不平行)
template<typename T>
point<T> lineIntersection(const Line<T>& a,
                          const Line<T>& b) {
  auto u = a.b - a.a, v = b.b - b.a;
  T d = cross(u, v);
  assert(d != 0);
  return a.a + u * (cross(b.a - a.a, v) / d);
}

// T 用浮點數；輸入依邊界排序的凸包，順逆時針皆可
// 可含邊界共線點；刪相鄰重複點，空/線段回傳 {0,{}}
// O(n)，回傳 {面積, 逆時針四角}，不改輸入
template<typename T>
pair<T, vector<point<T>>> minRectangleCover(
    vector<point<T>> p) {
  auto same = [](point<T> a, point<T> b) {
    return a.x == b.x && a.y == b.y;
  };
  p.erase(unique(p.begin(), p.end(), same), p.end());
  if (p.size() > 1 && same(p.front(), p.back()))
    p.pop_back();
  int n = p.size();
  if (n <= 2) return {0, {}};
  T signed_area = 0;
  for (int i = 0; i < n; ++i)
    signed_area += cross(p[i] - p[0],
                         p[(i + 1) % n] - p[0]);
  if (signed_area == 0) return {0, {}};
  if (signed_area < 0) reverse(p.begin(), p.end());
  T ans = numeric_limits<T>::max();
  vector<point<T>> corners;
  int top = 0, left = 0, right = 0;
  for (int i = 0; i < n; ++i) {
    auto e = p[(i + 1) % n] - p[i];
    auto height = [&](int k) {
      // line 到 p 圍成的四邊形面積
      return cross(e, p[k] - p[i]);
    };
    // 以當前點為原點，避免大座標的投影相減損失精度
    auto along = [&](int k) {
      return dot(e, p[k] - p[i]);
    };
    if (!i) {
      for (int k = 1; k < n; ++k) {
        if (height(k) > height(top)) top = k;
        if (along(k) < along(left)) left = k;
        if (along(k) > along(right)) right = k;
      }
    }
    while (height((top + 1) % n) >= height(top))
      top = (top + 1) % n;
    while (along((left + 1) % n) <= along(left))
      left = (left + 1) % n;
    while (along((right + 1) % n) >= along(right))
      right = (right + 1) % n;
    T lo = along(left), hi = along(right);
    T up = height(top), norm = square(e);
    T area = (hi - lo) * up / norm;
    if (area >= ans) continue;
    ans = area;
    auto at = [&](T x, T y) {
      return p[i] + (e * x + rot(e) * y) / norm;
    };
    corners = {at(lo, 0), at(hi, 0),
               at(hi, up), at(lo, up)};
  }
  return {ans, corners};
}
```

5.7 四邊形旋轉

```cpp
const long double PI = acos(-1);

// 寬 w 高 h 的四邊形，旋轉一個 pi 後在每個角度的寬高
// 回傳 freq + 1 筆等距取樣，非連續角度的精確極值
vector<pair<double, double>> rotate(double w, double h,
                                    int freq = 5) {
  // 自己調整精度
  assert(freq > 0);
  vector<pair<double, double>> res;
  res.reserve(freq + 1);
  for (int i = 0; i <= freq; ++i) {
    double theta = PI * i / freq;
    double c = fabs(cos(theta)), s = fabs(sin(theta));
    res.push_back({w * c + h * s, w * s + h * c});
  }
  return res;
}
```

5.8 旋轉

```cpp
const long double PI = acos(-1);
// 逆時針旋轉
// angle_red 為弧度
pair<double, double> rotate_point
    (double x, double y, double angle_rad) {
  angle_rad *= PI;
  double
       new_x = x * cos(angle_rad) - y * sin(angle_rad);
  double
       new_y = x * sin(angle_rad) + y * cos(angle_rad);
  return {new_x, new_y};
}

int main() {
  double x = 5, y = 0;
  double angle = 0.5;   // 逆時針旋轉 90 度
  auto result = rotate_point(x, y, angle);
  cout << result.first << " " << result.second << endl;
  // 0, 5
  return 0;
}
```

5.9 極座標轉直角座標

```cpp
// 極座標轉換為直角座標函數，theta 單位為弧度
const long double PI = acos(-1);
pair<double, double>
    > polar_to_cartesian(double r, double theta) {
  double theta_radians = theta * PI;
  double x = r * cos(theta_radians);
  double y = r * sin(theta_radians);
  return {x, y};
}

int main() {
  double r = 5, theta = 0.5;  // 極座標
  auto result = polar_to_cartesian(r, theta);
  cout << result.first << " " << result.second << endl;
  // 0, 5
  return 0;
}
```

5.10 直角座標轉極座標

```cpp
// 直角座標轉換為極座標
const long double PI = acos(-1);
std::pair<double
    , double> cartesian_to_polar(double x, double y) {
  double r = sqrt(x * x + y * y);
  double theta = atan2(y, x) / PI;
  return {r, theta};
}

int main() {
  double x = 3, y = 4;  // 直角座標
  auto result = cartesian_to_polar(x, y);
  cout << "r = " << result
       .first << ", theta = " << result.second << endl;
  // 5, 0.295167
  return 0;
}
```

6 Data Structure

6.1 Sparse Table

```cpp
class Sparse_Table {
  // 0-base
  // 要改成找最大把min換成max就好
  // 此版同時建 min/max；只需一種時可刪另一張表。
  vector<vector<int>> mn, mx;
public:
  Sparse_Table(const vector<int>& a) {
    int n = a.size(), k = n ? __lg(n) + 1 : 0;
    mn.assign(k, vector<int>(n));
    mx = mn;
    if (!n) return;
    mn[0] = mx[0] = a;
```

```
    for (int j = 1; j < k; ++j)
      for (int i = 0; i + (1 << j) <= n; ++i) {
        int r = i + (1 << (j - 1));
        mn[j][i] = min(mn[j-1][i], mn[j-1][r]);
        mx[j][i] = max(mx[j-1][i], mx[j-1][r]);
      }
  }
  // 非空詢問須滿足 0 <= l <= r < n。
  int query_min(int l, int r) const {
    if (l > r) return numeric_limits<int>::max();
    int j = __lg(r - l + 1);
    return min(mn[j][l], mn[j][r - (1 << j) + 1]);
  }
  int query_max(int l, int r) const {
    if (l > r) return numeric_limits<int>::lowest();
    int j = __lg(r - l + 1);
    return max(mx[j][l], mx[j][r - (1 << j) + 1]);
  }
};
```

6.2 Segement Tree

```
// 區間 add / assign / sum，單次 O(log n)
// 只有單點修改時，才可移除 push 與 lazy
// 最外層呼叫時，每個 id 都傳 1
// 大值搭配 #define int long long；加總須不溢位

const int N = 200000 + 9;
int a[N];
int seg[4 * N];
int lazy[4 * N];
bool has_assign[4 * N];

inline void pull(int id) {
    seg[id] = seg[id * 2] + seg[id * 2 + 1];
}

// tag：有 assign 時為指定值，否則為待加量
// assign 覆蓋舊操作，add 累加並保留 assign 標記
inline void apply(int id, int l, int r,
                  int v, bool assign = false) {
    if (assign) {
        seg[id] = v * (r - l + 1);
        lazy[id] = v;
        has_assign[id] = true;
    } else {
        seg[id] += v * (r - l + 1);
        lazy[id] += v;
    }
}

inline void push(int id, int l, int r) {
    if (l
        == r || (!has_assign[id] && !lazy[id])) return;
    int mid = (l + r) / 2;
    apply(id * 2, l, mid, lazy[id], has_assign[id]);
    apply(id * 2 + 1, mid + 1, r,
          lazy[id], has_assign[id]);
    lazy[id] = 0;
    has_assign[id] = false;
}

// 編號為 id 的節點，存的區間為 [l, r]
void build(int id, int l, int r) {
    lazy[id
        ] = 0; has_assign[id] = false; // 可重複 build
    // 葉節點的值
    if (l == r) { seg[id] = a[l]; return; }
    int mid = (l + r) / 2; // 將區間切成兩半
    build(id * 2, l, mid);                  // 左子節點
    build(id * 2 + 1, mid + 1, r);          // 右子節點
    pull(id);
}

// 區間查詢：回傳 [ql, qr] 的區間和
int query(int id, int l, int r, int ql, int qr) {
    if (r < ql || qr < l) return 0;         // 交集為空
    if (ql <= l && r <= qr) return seg[id]; // 完全覆蓋
    push(id, l, r); // 下傳 lazy
    int mid = (l + r) / 2;
    return query(id * 2, l, mid, ql, qr)    // 左
         + query(id * 2 + 1, mid + 1, r, ql, qr); // 右
    // 否則，往左、右進行遞迴
}

// 共用更新：assign=true 設值，false 加值
void range_update(int id, int l, int r,
                  int ql, int qr, int x, bool assign) {
    if (r < ql || qr < l) return; // 交集為空
    // 完全覆蓋
    if (ql <= l && r <= qr) {
        apply(id, l, r, x, assign);
        return;
    }
    push(id, l, r); // 下傳 lazy 再往下走
    int mid = (l + r) / 2;
    range_update(id * 2, l, mid, ql, qr, x, assign);
    range_update(id * 2 + 1, mid + 1, r,
                 ql, qr, x, assign);
    pull(id);
}

// 區間加值：將 [ql, qr] 每個位置都加上 x
void range_add(int id, int l, int r,
               int ql, int qr, int x) {
    range_update(id, l, r, ql, qr, x, false);
}
// 區間設值：將 [ql, qr] 每個位置都改成 x（含 0）
void range_assign(int id, int l, int r,
                  int ql, int qr, int x) {
    range_update(id, l, r, ql, qr, x, true);
}
// 單點修改 (設值版)：將 a[i] 改成 x
void modify(int id, int l, int r, int i, int x) {
    range_assign(id, l, r, i, i, x);
}

// 同一棵樹交錯使用（先令 a[1..4] = {1,2,3,4}）：
// build(1, 1, 4);
// range_add(1, 1, 4, 1, 3, 2);    // {3,4,5,4}
// range_assign(1, 1, 4, 2, 4, 0); // {3,0,0,0}
// range_add(1, 1, 4, 3, 4, -1);   // {3,0,-1,-1}
// query(1, 1, 4, 1, 4);           // 區間和 = 1

// ================================================
// 區間加值 / 設值 + 區間最小值 + 最小值出現次數
// 以下替換主模板對應部分，不要整段接在後面
// 1. 宣告：
//    int seg[4 * N];
//    -> pair<int,int> seg[4 * N];
//    lazy / has_assign 宣告不變
// 2. 葉子：
//    build 的 seg[id] = a[l];
//    -> seg[id] = {a[l], 1};
//    modify 已共用 range_assign，不用改
// 3. pull / query 的合併：抽出 merge，
//    min/cnt 邏輯只寫一次（pull 直接呼叫）：
inline pair<int, int> merge(pair<int, int> L,
                            pair<int, int> R) {
    // cnt 為 0 代表空區間，不需 INF 哨兵
    if (!L.second) return R;
    if (!R.second) return L;
    // min 較小者整包保留
    if (L.first != R.first)
        return L.first < R.first ? L : R;
    // 相等時累加個數
    return {L.first, L.second + R.second};
}
inline void pull(int id) {
    seg[id] = merge(seg[id * 2], seg[id * 2 + 1]);
}
// 4. apply：加值保留 cnt；設值讓整段都成為最小值
inline void apply(int id, int l, int r,
                  int v, bool assign = false) {
    if (assign) {
        seg[id] = {v, r - l + 1};
        lazy[id] = v;
        has_assign[id] = true;
    } else {
        seg[id].first += v;
        lazy[id] += v;
    }
}
// query 骨架不變，只有回傳型別、
// 空區間值、最後的合併改用 merge：
```

```
pair<int, int> query(int id, int l, int r,
                     int ql, int qr) {
    if (r < ql || qr < l) return {0, 0};
    if (ql <= l && r <= qr) return seg[id];
    push(id, l, r);
    int mid = (l + r) / 2;
    return merge(query(id * 2, l, mid, ql, qr),
                 query(id * 2 + 1, mid + 1, r,
                       ql, qr));
}
// 5. push / 更新函數完全不用改（build 只改葉子值）
//    沿用上方 add / assign / add 範例，最後
//    query(1, 1, 4, 1, 4) = {-1, 2}。
```

6.3 Li Chao Tree

```
// 用途：維護一組直線 y = m*x + b，
//       支援「加入直線」與「單點查詢最大值」。
// 求最小值：對所有 (m,b) 取負後 query 再取負即可。
// 適用情境：動態 CHT、斜率優化 DP。
// m*x+b 須不溢位 ll，且介於 -LINF 與 LINF。
using ll = long long;
const ll LINF = 4e18; // 大於所有可能的直線取值

struct Line {
    ll m, b; // y = m*x + b
    ll eval(ll x) const { return m * x + b; }
};

struct LiChao {
    vector<ll> xs;
    vector<Line> tree;
    int n;

    LiChao(const vector<ll>& qs) {
        xs = qs;
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()),
                 xs.end());
        n = (int)xs.size();
        tree.
            assign(4 * max<int>(1, n), Line{0, -LINF});
    }

    void add_line(Line nw, int id, int l, int r) {
        int m = (l + r) / 2;
        bool lef =
            nw.eval(xs[l]) > tree[id].eval(xs[l]);
        bool mid =
            nw.eval(xs[m]) > tree[id].eval(xs[m]);
        if (mid) swap(tree[id], nw);
        if (r - l == 1) return;
        if (lef != mid) add_line(nw, id * 2, l, m);
        else add_line(nw, id * 2 + 1, m, r);
    }

    void add_line(ll m, ll b) {
        if (n == 0) return;
        add_line(Line{m, b}, 1, 0, n);
    }

    ll query(ll x, int id, int l, int r) const {
        ll res = tree[id].eval(x);
        if (r - l == 1) return res;
        int m = (l + r) / 2;
        if (x < xs[m])
            return max(res, query(x, id * 2, l, m));
        else
            return max(res, query(x, id * 2 + 1,
                                  m, r));
    }

    ll query(ll x) const {
        if (n == 0) return -LINF;
        auto it =
            lower_bound(xs.begin(), xs.end(), x);
        int idx = (int)(it - xs.begin());
        assert(idx < n && xs[idx] == x);
        return query(x, 1, 0, n);
    }
};
//  vector<ll> qs = {1, 3, 5};  // 所有會 query 的點
//  LiChao lc(qs);
//  lc.add_line(2, -1);         // y = 2x - 1
//  lc.add_line(-3, 7);         // y = -3x + 7
//  cout << lc.query(3) << '\n'; // 兩線在 x=3 的最大值
```

6.4 動態開點線段樹

```
using ll = long long;
struct Sum {
  static constexpr ll id = 0;
  static ll merge(ll a, ll b) { return a + b; }
  static ll add(ll a, ll x, ll len) {
    return a + x * len;
  }
  static ll assign(ll x, ll len) { return x * len; }
};
struct Max {
  static constexpr ll id = LLONG_MIN;
  static ll merge(ll a, ll b) { return max(a, b); }
  static ll add(ll a, ll x, ll) { return a + x; }
  static ll assign(ll x, ll) { return x; }
};
struct Min {
  static constexpr ll id = LLONG_MAX;
  static ll merge(ll a, ll b) { return min(a, b); }
  static ll add(ll a, ll x, ll) { return a + x; }
  static ll assign(ll x, ll) { return x; }
};

// 閉區間，初始全 0；區間長度及運算須在 ll 範圍內
// 支援區間加值、區間賦值、單點設值、區間查詢
// 每次 O(log C)，Q 次更新空間 O(Q log C)
// C = R-L+1；query 不開點；使用 C++17
template<class Op> struct DynamicSegTree {
  struct Node {
    int32_t lc = 0, rc = 0;
    ll val = 0, lazy
         = 0; // lazy：要給子樹的值，set 決定覆寫或累加
    bool set = false;
  };
  ll L, R;
  vector<Node> t = {{}, {}}; // 0 空節點，1 根

  DynamicSegTree(ll l, ll r) : L(l), R(r) {
    // 用 unsigned 差值
        檢查長度，避免有號溢位，亦支援 32-bit 編譯器。
    assert(l <= r &&
         (unsigned long long)r - (unsigned long long)l
                        < (unsigned
                            long long)LLONG_MAX);
  }
  void range_add(ll l, ll r, ll x) {
    check(l, r);
    if (x) update(1, L, R, l, r, x, false);
  }
  void range_assign(ll l, ll r, ll x) {
    check(l, r);
    update(1, L, R, l, r, x, true);
  }
  void modify(ll i, ll x) {
    check(i, i);
    update(1, L, R, i, i, x, true);
  }
  ll query(ll l, ll r) const {
    check(l, r);
    return query(1, L, R, l, r, 0, false);
  }
private:
  void check(ll l, ll r) const {
    assert(L <= l && l <= r && r <= R);
  }
  int32_t make() {
    t.push_back({});
    return static_cast<int32_t>(t.size() - 1);
  }
  void apply_add(int32_t p, ll l, ll r, ll x) {
    t[p].val = Op::add(t[p].val, x, r-l+1);
    t[p].lazy += x; // set
        為真時即「賦值再加」，lazy 是子樹要變成的新值
  }
  void apply_assign(int32_t p, ll l, ll r, ll x) {
    t[p].val = Op::assign(x, r-l+1);
    t[p].lazy = x;
    t[p].set = true;
  }
  void push(int32_t p, ll l, ll r) {
    ll x = t[p].lazy, m = l + (r-l)/2;
    if ((!x && !t[p].set) || l == r) return;
```

```
    if (!t[p].lc) t[p].lc = make();
    if (!t[p].rc) t[p].rc = make();
    if (t[p].set) {
      apply_assign(t[p].lc, l, m, x);
      apply_assign(t[p].rc, m+1, r, x);
      t[p].set = false;
    } else {
      apply_add(t[p].lc, l, m, x);
      apply_add(t[p].rc, m+1, r, x);
    }
    t[p].lazy = 0;
  }
  int32_t update(int32_t p, ll l, ll r,
                 ll a, ll b, ll x, bool assign) {
    if (!p) p = make();
    if (a <= l && r <= b) {
      if (assign) apply_assign(p, l, r, x);
      else apply_add(p, l, r, x);
      return p;
    }
    push(p, l, r);
    ll m = l + (r-l)/2;
    if (a <= m)
      t[p].lc = update(t[p].lc, l, m, a, b, x, assign);
    if (b > m)
      t[p].rc
           = update(t[p].rc, m+1, r, a, b, x, assign);
    t[p].val = Op::merge(t[t[p].lc].val,
                         t[t[p].rc].val);
    return p;
  }
  ll query(int32_t p, ll l, ll r,
           ll a, ll
                b, ll carry, bool carried_set) const {
    if (r < a || b < l) return Op::id;
    ll len = min(r,b)-max(l,a)+1;
    if (!p)
      return carried_set ? Op::assign(carry, len)
                         : Op::add(0, carry, len);
    if (a <= l && r <= b)
      return carried_set ? Op::assign(carry, len)
                         : Op::add
                             (t[p].val, carry, r-l+1);
    ll m = l + (r-l)/2;
    if (!carried_set) {
      // 祖先的加值較新，加在子孫的標記值之上；
      // 已帶著較新的 assign 時，子孫較舊的標記一律忽略
      carry += t[p].lazy;
      if (t[p].set) carried_set = true;
    }
    return Op::merge(
      query(t[p].lc, l, m, a, b, carry, carried_set),
      query
          (t[p].rc, m+1, r, a, b, carry, carried_set));
  }
};

// DynamicSegTree<Max> seg(1, 1000000000); // Sum/Min
// seg.range_add(2, 4, 3); seg.range_assign(2, 3, 5);
// seg.modify(3, 5); seg.query(1, 5);
```

6.5 Discrete Segement Tree

```
#include <bits/stdc++.h>
#define int long long
#define de(x) cout << #x << '=' << x << ", "
#define dd cout << '\n';

/*
本題給n個長方形 0 <= x1, y1, x2, y2 <= 10^9
求被奇數個長方形覆蓋的面積

想法：掃描線+離散化線段樹
區間xor加值 + 區間sum查詢
*/

/*
先將輸入點離散化 [a1, a2, ... , an] => [1, 2, ... , n]
將每個相鄰離散點區間重新編號 [1, 2] => 編
    號1  [2, 3] => 編號 2 , ... , [n - 1, n] 編號 n - 1
將每個區間的長度都記下 pre[1] = a2 - a1
     , pre[2] = a3 - a2 , ... , pre[n - 1] = an - an-1
若要加值區間[a, b]，則加線段樹上的區間[a, b - 1]
和一般線段樹唯一差別在 void apply
    ()，區間長度從(r - l + 1)變成(pre[r] - pre[l - 1])
*/

/*

testcase 1 :
2
0 0 4 4
1 1 3 3

answer : 12

testcase 2 :
4
0 0 10 10
1 1 11 11
2 2 12 12
3 3 13 13

answer : 72

*/

using namespace std;
int tt = 1;

int a[600009];
int seg[4 * 600009];
int lazy[4 * 600009];
int mp[600009];
int pre[600009];

map<int, int> mp2;

void pull(int id){
    seg[id] = seg[id * 2] + seg[id * 2 + 1];
}
void apply(int id, int l, int r, int v){
    seg[id] = (pre[r] - pre[l - 1]) - seg[id];
    lazy[id] ^= 1;
}

void push(int id, int l, int r){
    if(!lazy[id] || l == r) return;
    int mid = (l + r) / 2;
    apply(id * 2, l, mid, lazy[id]);
    apply(id * 2 + 1, mid + 1, r, lazy[id]);
    lazy[id] = 0;
}

int query(int id, int l, int r, int ql, int qr){
    if(r < ql || qr < l) return 0;
    if(ql <= l && r <= qr) return seg[id];
    push(id, l, r);
    int mid = (l + r) / 2;
    return query(id * 2, l, mid, ql
        , qr) + query(id * 2 + 1, mid + 1, r, ql, qr);
}

void range_add
    (int id, int l, int r, int ql, int qr, int x){
    if(r < ql || qr < l) return ;
    if(ql <= l && r <= qr) {
        apply(id, l, r, x);
        return;
    }
    push(id, l, r);
    int mid = (l + r) / 2;
    range_add(id * 2, l, mid, ql, qr, x);
    range_add(id * 2 + 1, mid + 1, r , ql, qr, x);
    pull(id);
}

void _pre(){
    cout.tie(nullptr);
    //cin >> tt;
}

struct rec{
    int loc, down, top;
};

bool comp(rec a, rec b){
    return a.loc < b.loc;
}
```

```
void swap(int &a, int &b){
    int tmp = a;
    a = b;
    b = tmp;
}

void solve(){
    int n, a, b, c, d, i, j, k;
    vector<rec> v;
    cin >> n;
    set<int> st;
    for(i = 1; i <= n; i ++){
        cin >> a >> b >> c >> d;
        int high = max(b, d);
        int low = min(b, d);
        v.push_back({a, low, high});
        v.push_back({c, low, high});
        st.insert(low);
        st.insert(high);
    }
    int N = 0;
    pre[0] = 0;
    mp[0] = 0;
    vector<int> vv;

    for(auto it = st.begin(); it != st.end(); it ++){
        mp[++ N] = *it;
        mp2[*it]= N;
        vv.push_back(*it);
    }

    for(i = 0; i < (int)vv.size() - 1; i ++){
      pre[i + 1] = pre[i] + (vv[i + 1] - vv[i]);
    }

    for(i = 0; i < (int)v.size(); i ++){
        v[i] = {v[
            i].loc, mp2[v[i].down], mp2[v[i].top] - 1};
    }
    sort(v.begin(), v.end(), comp);
    int ans = 0;

    int lastloc = v[0].loc;
    range_add(1, 1, N - 1, v[0].down, v[0].top, 1);

    for(i = 1; i < (int)v.size(); i ++){
        int down = v[i].down;
        int top = v[i].top;
        int loc = v[i].loc;

        int q = query(1, 1, N - 1, 1, N - 1);

        ans += (loc - lastloc) * q;
        range_add(1, 1, N - 1, down, top, 1);
        lastloc = loc;
    }

    cout << ans << '\n';
}
signed main(){
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    _pre();
    while(tt--){
        solve();
    }
}
```

6.6 Link Cut Tree

```
// 通常用於對樹上任兩點間的路徑做加值、修改、查詢等工作
// 與線段樹相同，要修改 LCT 的功能只需更改
// pull、push、fix、query 等函數，再加上需要的懶標即可
// 範例為樹上任兩點 x, y 路徑上的權值 xor
// 和，樹上任意點單點改值
const int N = 300005;
class LinkCutTree {
private:
#define lc(x) node[x].ch[0]
#define rc(x) node[x].ch[1]
#define fa(x) node[x].fa
#define rev(x) node[x].rev
#define val(x) node[x].val
#define sum(x) node[x].sum
  struct Tree {
    int val, sum, fa, rev, ch[2];
  } node[N];
  inline void pull(int x) {
    sum(x) = val(x) ^ sum(lc(x)) ^ sum(rc(x));
  }
  inline void reverse(int x) {
    swap(lc(x), rc(x));
    rev(x) ^= 1;
  }
  inline void push(int x) {
    if (rev(x)) {
      reverse(lc(x));
      reverse(rc(x));
      rev(x) ^= 1;
    }
  }
  inline bool get(int x) { return rc(fa(x)) == x; }
  inline bool isroot(int x) {
    return (lc(fa(x)) ^ x) && (rc(fa(x)) ^ x);
  }
  inline void update(int x) {
    if (!isroot(x)) update(fa(x));
    push(x);
  }
  void rotate(int x) {
    int y = fa(x), z = fa(y), d = get(x);
    if (!isroot(y))
      node[z].ch[get(y)] = x; // 重要，不能更換順序
    fa(x) = z;
    node[fa(node[x].ch[d ^ 1]) = y].ch[d] =
      node[x].ch[d ^ 1];
    node[fa(y) = x].ch[d ^ 1] = y;
    pull(y), pull(x); // 先 y 再 x
  }
  void splay(int x) {
    update(x);
    for (int y = fa(x); !isroot(x);
         rotate(x), y = fa(x)) {
      if (!isroot(y)) rotate(get(x) == get(y) ? y : x);
    }
    pull(x);
  }
  int access(int x) {
    int p = 0;
    for (; x; x = fa(p = x)) {
      splay(x), rc(x) = p, pull(x);
    }
    return p;
  }
  inline void makeroot(int x) {
    access(x), splay(x), reverse(x);
  }
  inline int findroot(int x) {
    access(x), splay(x);
    while (lc(x)) { push(x), x = lc(x); }
    return splay(x), x;
  }
  inline void split(int x, int y) {
    makeroot(x), access(y), splay(y);
  }

public:
  inline void init(int len, int *data) {
    for (int i = 1; i <= len; ++i) {
      node[i].val = data[i];
    }
  }
  inline void link(int x, int y) { // 連邊
    makeroot(x);
    if (findroot(y) == x) return;
    fa(x) = y;
  }
  inline void cut(int x, int y) { // 斷邊
    makeroot(x);
    if (findroot(y) != x || fa(y) != x || lc(y))
      return;
    fa(y) = rc(x) = 0;
    pull(x);
  }
  inline void fix(int x, int v) { // 單點改值
    splay(x);
    val(x) = v;
  }
  // 區間查詢
  inline int query(int x, int y) {
```

```
    return split(x, y), sum(y);
  }
};

LinkCutTree LCT;
int n, a[N];

signed main() {
  int n, q, op, x, y;
  cin >> n >> q;
  for (int i = 1; i <= n; ++i) { cin >> a[i]; }
  LCT.init(n, a);
  while (q--) {
    cin >> op >> x >> y;
    if (op == 0) {
      cout << LCT.query(x, y) << endl;
    } else if (op == 1) {
      LCT.link(x, y);
    } else if (op == 2) {
      LCT.cut(x, y);
    } else {
      LCT.fix(x, y);
    }
  }
  return 0;
}
```

6.7 BIT

```
#define lowbit(x) ((x) & -(x))
// 1-based，bit 開 n + 1 格；modify 的 idx 須 >= 1。
void modify(vector<int>& bit, int idx, int val) {
  for (; idx < (int)bit.size(); idx += lowbit(idx))
    bit[idx] += val;
}

int query(const vector<int>& bit, int idx) {
  int ans = 0;
  for (; idx > 0; idx -= lowbit(idx)) ans += bit[idx];
  return ans;
}

// the first i s.t. a[1]+...+a[i] >= k
// 須 a[i] >= 0、k >= 1；若總和不足，回傳 n + 1。
int findK(const vector<int>& bit, int k) {
  int n = (int)bit.size() - 1, idx = 0;
  for (int step = n ? (1LL << __lg(n)) : 0;
       step; step >>= 1) {
    int next = idx + step;
    if (next <= n && bit[next] < k)
      idx = next, k -= bit[next];
  }
  return idx + 1;
}

//O(n)建bit
// a 也是 1-based，第 0 格不使用。
vector<int> buildBIT(const vector<int>& a) {
  auto bit = a;
  for (int i = 1; i < (int)bit.size(); ++i) {
    int j = i + lowbit(i);
    if (j < (int)bit.size()) bit[j] += bit[i];
  }
  return bit;
}
```

6.8 2D BIT

```
//2維BIT
#define lowbit(x) (x&-x)

class BIT {
    int n;
    vector<int> bit;

public:
    void init(int _n) {
        n = _n;
        bit.resize(n + 1);
        for(auto &b : bit) b = 0;
    }
    int query(int x) const {
        int sum = 0;
        for(; x; x -= lowbit(x))
            sum += bit[x];
        return sum;
    }
    void modify(int x, int val) {
        for(; x <= n; x += lowbit(x))
            bit[x] += val;
    }
};

class BIT2D {
    int m;
    vector<BIT> bit1D;

public:
    void init(int _m, int _n) {
        m = _m;
        bit1D.resize(m + 1);
        for(auto &b : bit1D) b.init(_n);
    }
    int query(int x, int y) const {
        int sum = 0;
        for(; x; x-= lowbit(x))
            sum += bit1D[x].query(y);
        return sum;
    }
    void modify(int x, int y, int val) {
        for(; x <= m; x += lowbit(x))
            bit1D[x].modify(y,val);
    }
};
```

6.9 undo DSU

```
struct dsu_undo{
  vector<int>sz,p;
  int comps;
  dsu_undo(int n){
    sz.assign(n+5,1);
    p.resize(n+5);
    for(int i = 1;i<=n;++i)p[i] = i;
    comps = n;
  }
  vector<pair<int,int>>opt;
  int Find(int x){
    return x==p[x]?x:Find(p[x]);
  }
  bool Union(int a,int b){
    int pa = Find(a),pb = Find(b);
    if(pa==pb)return 0;
    if(sz[pa]<sz[pb])swap(pa,pb);
    sz[pa]+=sz[pb];
    p[pb] = pa;
    opt.push_back({pa,pb});
    comps--;
    return 1;
  }
  void undo(){
        auto [pa,pb] = opt.back();
        opt.pop_back();
        p[pb] = pb;
        sz[pa]-=sz[pb];
        comps++;
  }
};
```

6.10 Treap

```
// author: (ttzytt)[ttzytt.com]
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <tuple>
using namespace std;

struct Node {
  Node *ch[2];
  int val, prio;
  int cnt;
  int siz;

  Node(int _val) : val(_val), cnt(1), siz(1) {
    ch[0] = ch[1] = nullptr;
    prio = rand();
  }

  Node(Node *_node) {
```

```
    val = _node->val, prio = _node
        ->prio, cnt = _node->cnt, siz = _node->siz;
  }

  void upd_siz() {
    siz = cnt;
    if (ch[0] != nullptr) siz += ch[0]->siz;
    if (ch[1] != nullptr) siz += ch[1]->siz;
  }
};

struct none_rot_treap {
#define _3 second.second
#define _2 second.first
  Node *root;

  pair<Node *, Node *> split(Node *cur, int key) {
    if (cur == nullptr) return {nullptr, nullptr};
    if (cur->val <= key) {
      auto temp = split(cur->ch[1], key);
      cur->ch[1] = temp.first;
      cur->upd_siz();
      return {cur, temp.second};
    } else {
      auto temp = split(cur->ch[0], key);
      cur->ch[0] = temp.second;
      cur->upd_siz();
      return {temp.first, cur};
    }
  }

  tuple<Node *,
       Node *, Node *> split_by_rk(Node *cur, int rk) {
    if (cur
        == nullptr) return {nullptr, nullptr, nullptr};
    int ls_siz
         = cur->ch[0] == nullptr ? 0 : cur->ch[0]->siz;
    if (rk <= ls_siz) {
      Node *l, *mid, *r;
      tie(l, mid, r) = split_by_rk(cur->ch[0], rk);
      cur->ch[0] = r;
      cur->upd_siz();
      return {l, mid, cur};
    } else if (rk <= ls_siz + cur->cnt) {
      Node *lt = cur->ch[0];
      Node *rt = cur->ch[1];
      cur->ch[0] = cur->ch[1] = nullptr;
      return {lt, cur, rt};
    } else {
      Node *l, *mid, *r;
      tie(l, mid, r) = split_by_rk
          (cur->ch[1], rk - ls_siz - cur->cnt);
      cur->ch[1] = l;
      cur->upd_siz();
      return {cur, mid, r};
    }
  }

  Node *merge(Node *u, Node *v) {
    if (u == nullptr && v == nullptr) return nullptr;
    if (u != nullptr && v == nullptr) return u;
    if (v != nullptr && u == nullptr) return v;
    if (u->prio < v->prio) {
      u->ch[1] = merge(u->ch[1], v);
      u->upd_siz();
      return u;
    } else {
      v->ch[0] = merge(u, v->ch[0]);
      v->upd_siz();
      return v;
    }
  }

  void insert(int val) {
    auto temp = split(root, val);
    auto l_tr = split(temp.first, val - 1);
    Node *new_node;
    if (l_tr.second == nullptr) {
      new_node = new Node(val);
    } else {
      l_tr.second->cnt++;
      l_tr.second->upd_siz();
    }
    Node *l_tr_combined =
        merge(l_tr.first, l_tr.second
             == nullptr ? new_node : l_tr.second);
    root = merge(l_tr_combined, temp.second);
  }

  void del(int val) {
    auto temp = split(root, val);
    auto l_tr = split(temp.first, val - 1);
    if (l_tr.second->cnt > 1) {
      l_tr.second->cnt--;
      l_tr.second->upd_siz();
      l_tr.first = merge(l_tr.first, l_tr.second);
    } else {
      if (temp.first == l_tr.second) {
        temp.first = nullptr;
      }
      delete l_tr.second;
      l_tr.second = nullptr;
    }
    root = merge(l_tr.first, temp.second);
  }

  int qrank_by_val(Node *cur, int val) {
    auto temp = split(cur, val - 1);
    int ret = (temp
        .first == nullptr ? 0 : temp.first->siz) + 1;
    root = merge(temp.first, temp.second);
    return ret;
  }

  int qval_by_rank(Node *cur, int rk) {
    Node *l, *mid, *r;
    tie(l, mid, r) = split_by_rk(cur, rk);
    int ret = mid->val;
    root = merge(merge(l, mid), r);
    return ret;
  }

  int qprev(int val) {
    auto temp = split(root, val - 1);
    int ret
         = qval_by_rank(temp.first, temp.first->siz);
    root = merge(temp.first, temp.second);
    return ret;
  }

  int qnex(int val) {
    auto temp = split(root, val);
    int ret = qval_by_rank(temp.second, 1);
    root = merge(temp.first, temp.second);
    return ret;
  }
};

none_rot_treap tr;

int main() {
  srand(time(nullptr));
  int t;
  scanf("%d", &t);
  while (t--) {
    int mode;
    int num;
    scanf("%d%d", &mode, &num);
    switch (mode) {
      case 1:
        tr.insert(num);
        break;
      case 2:
        tr.del(num);
        break;
      case 3:
        printf("%d\n", tr.qrank_by_val(tr.root, num));
        break;
      case 4:
        printf("%d\n", tr.qval_by_rank(tr.root, num));
        break;
      case 5:
        printf("%d\n", tr.qprev(num));
        break;
      case 6:
        printf("%d\n", tr.qnex(num));
        break;
    }
  }
}
```

6.11 Treap 區間操作

```cpp
// author: (ttzytt)[ttzytt.com]
#include <cstdlib>
#include <ctime>
#include <iostream>
using namespace std;

// 參考：https
    ://www.cnblogs.com/Equinox-Flower/p/10785292.html
struct Node {
  Node* ch[2];
  int val, prio;
  int cnt;
  int siz;
  bool to_rev =
      false;  // 需要把这个子树下的每一个节点都翻转过来

  Node(int _val) : val(_val), cnt(1), siz(1) {
    ch[0] = ch[1] = nullptr;
    prio = rand();
  }

  int upd_siz() {
    siz = cnt;
    if (ch[0] != nullptr) siz += ch[0]->siz;
    if (ch[1] != nullptr) siz += ch[1]->siz;
    return siz;
  }

  void pushdown() {
    swap(ch[0], ch[1]);
    if (ch[0] != nullptr) ch[0]->to_rev ^= 1;
    // 如果原来子节点也要翻转，
        那两次翻转就抵消了，如果子节点不翻转，那这个
    //  tag 就需要继续被 push 到子节点上
    if (ch[1] != nullptr) ch[1]->to_rev ^= 1;
    to_rev = false;
  }

  void check_tag() {
    if (to_rev) pushdown();
  }
};

struct Seg_treap {
  Node* root;
#define siz(_) (_ == nullptr ? 0 : _->siz)

  pair<Node*, Node*> split(Node* cur, int sz) {
    // 按照树的大小划分
    if (cur == nullptr) return {nullptr, nullptr};
    cur->check_tag();
    if (sz <= siz(cur->ch[0])) {
      // 左边的子树就够了
      auto temp = split(cur->ch[0], sz);
      // 左边
          的子树不一定全部需要，temp.second 是不需要的
      cur->ch[0] = temp.second;
      cur->upd_siz();
      return {temp.first, cur};
    } else {
      // 左边
          的加上右边的一部分（当然也包括这个节点本身）
      auto temp =
          split(cur->ch[1], sz - siz(cur->ch[0]) - 1);
      cur->ch[1] = temp.first;
      cur->upd_siz();
      return {cur, temp.second};
    }
  }

  Node* merge(Node* sm, Node* bg) {
    // small, big
    if (sm == nullptr && bg == nullptr) return nullptr;
    if (sm != nullptr && bg == nullptr) return sm;
    if (sm == nullptr && bg != nullptr) return bg;
    sm->check_tag(), bg->check_tag();
    if (sm->prio < bg->prio) {
      sm->ch[1] = merge(sm->ch[1], bg);
      sm->upd_siz();
      return sm;
    } else {
      bg->ch[0] = merge(sm, bg->ch[0]);
      bg->upd_siz();
      return bg;
    }
  }

  void insert(int val) {
    auto temp = split(root, val);
    auto l_tr = split(temp.first, val - 1);
    Node* new_node;
    if (l_tr
        .second == nullptr) new_node = new Node(val);
    Node* l_tr_combined =
        merge(l_tr.first, l_tr.second
              == nullptr ? new_node : l_tr.second);
    root = merge(l_tr_combined, temp.second);
  }

  void seg_rev(int l, int r) {
    // 这里的 less 和 more 是相对于 l 的
    auto less = split(root, l - 1);
    // 所有小于等于 l - 1 的会在 less 的左边
    auto more = split(less.second, r - l + 1);
    // 拿出从 l 开始的前 r - l + 1 个
    more.first->to_rev = true;
    root = merge
        (less.first, merge(more.first, more.second));
  }

  void print(Node* cur) {
    if (cur == nullptr) return;
    cur->check_tag();
    print(cur->ch[0]);
    cout << cur->val << " ";
    print(cur->ch[1]);
  }
};

Seg_treap tr;

int main() {
  srand(time(nullptr));
  int n, m;
  cin >> n >> m;
  for (int i = 1; i <= n; i++) tr.insert(i);
  while (m--) {
    int l, r;
    cin >> l >> r;
    tr.seg_rev(l, r);
  }
  tr.print(tr.root);
}
```

7 Dynamic Programing

7.1 LCS

```cpp
// O(n m) 時間、O(min(n,m)) 空間，兩字串皆 0-based。
int LCS(const string& t1, const string& t2) {
  string_view a = t1, b = t2;
  if (a.size() < b.size()) swap(a, b);
  vector<int> dp(b.size() + 1);
  for (char c : a) {
    int prev = 0;
    for (int j = 1; j <= (int)b.size(); ++j) {
      int old = dp[j];
      dp[j] = c == b[j-1] ? prev + 1
                          : max(dp[j], dp[j-1]);
      prev = old;
    }
  }
  return dp.back();
}

// 匹配位置轉 LIS；同字元的位置必須逆序。
// R = 相等字元對數；O(n+m+R log(min(n,m)+1))。
// R 可達 n*m，重複字元多時建議用上面的 DP。
int LCS_sparse(const string& a, const string& b) {
  vector<vector<int>> pos(256);
  for (int i = (int)a.size() - 1; i >= 0; --i)
    pos[(unsigned char)a[i]].push_back(i);
  vector<int> tail;
  for (unsigned char c : b) for (int p : pos[c]) {
    auto it = lower_bound(tail.begin(), tail.end(), p);
    if (it == tail.end()) tail.push_back(p);
    else *it = p;
  }
  return tail.size();
}
```

```
}
```

7.2 LIS

```
int LIS(vector<int>& save) {
  vector<int> dp;
  int n = save.size();
  for (int i = 0; i < n; i++) {
    auto it = lower_bound(dp.begin(),dp.end(),save[i]);
    if(it == dp.end()) dp.push_back(save[i]);
    else *it = save[i];
  }
  return dp.size();
}
```

7.3 Knapsack

```
/**
 * 背包問題：
 * 1. dp[i][j]: 考慮 1~i 個物品，重量為 j 時的最大價格
 * 2. dp[i][j]: 考慮 1~i 個物品，價值為 j 時的最小重量
 */

// 當重量比較輕時 O(nw)
vector<int> dp(sum + 1, 0);
for (int i = 1; i <= n; ++i) {
  for (int j = sum /* bound */; j >= weight[i]; --j) {
    if (dp[j] < dp[j - weight[i]] + price[i]) {
      dp[j] = dp[j - weight[i]] + price[i];
      backtrack[i][j] = 1;
    }
  }
}
// 當重量比較重時 O(nc)
vector<int> dp(sum + 1, 1e9 + 7);
dp[0] = 0;
for (int i = 1; i <= n; ++i) {
  for (int j = sum /* bound */; j >= price[i]; --j) {
    if (dp[j] > dp[j - price[i]] + weight[i]) {
      dp[j] = dp[j - price[i]] + weight[i];
      backtrack[i][j] = 1;
    }
  }
}

// backtrack: 找到當 bound 為 k 時，背包內有哪些東西
// 註：只找到其中一種
int l = n, r = k;
vector<int> ans;
while (l != 0 && r != 0) {
  if (backtrack[l][r]) {
    ans.push_back(l);
    r -= weight[l]; // 當用方法一時，用這行
    r -= price[l]; // 當用方法二時，用這行
  }
  l--;
}
```

7.4 位元 dp

```
// SOS DP：子集合和，O(k * 2^k)，額外空間 O(1)
// f：大小為 2^k（k >= 0），原地更新
// f[mask]：mask 所有子集合的原始值總和
//   包含空集合與 mask 本身；加總須不溢位
// 用法：auto f = a; sosdp(f);
//   a = {1, 2, 3, 4}，結果 f = {1, 3, 4, 10}
// 取模時將 += 改成模加法；bit 必須放外層
template<typename T>
void sosdp(vector<T>& f) {
  size_t n = f.size();
  assert(n && !(n & (n - 1)));
  for (size_t bit = 1; bit < n; bit <<= 1)
    for (size_t mask = 0; mask < n; ++mask)
      if (mask & bit) f[mask] += f[mask ^ bit];
}

// 位元操作（long long，0 <= n < 63）
// 檢查第 n 位是否為1：a & (1LL << n)
// 強制將第 n 位變成1：a |= (1LL << n)
// 強制將第 n 位變成0：a &= ~(1LL << n)
// 將第 n 位反轉(1變0, 0變1)：a ^= (1LL << n)
// 第 0 ~ n - 1位 全部都是1：(1LL << n) - 1
// 兩個集合的聯集：S = a | b
// 兩個集合的交集：S = a & b
```

7.5 經典 dp 轉移式

```
/*
最大區間和:

dp[i] 代表 由第 i 項結尾時的最大區間和
dp[0] = arr[0]
dp[i] = max(dp[i - 1], arr[i])
ans = max_element(dp)

*/
```

7.6 編輯距離

```
void editDistance() {
    string s1, s2; cin >> s1 >> s2;
    int n1 = s1.size(), n2 = s2.size();
    vector<int> dp(n2 + 1);
    iota(dp.begin(), dp.end(), 0);
    for (int i = 1; i <= n1; i++) {
        vector<int> cur(n2 + 1); cur[0] = i;
        for (int j = 1; j <= n2; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                cur[j] = dp[j - 1];
            } else {
                // s1 新增等價於 s2 砍掉
                // dp[i
                    ][j] = min(s2 新增, 修改, s1 新增);
                cur[j] = min({
                    cur[j - 1], dp[j - 1], dp[j]}) + 1;
            }
        }
        swap(dp, cur);
    }
    cout << dp[n2] << "\n";
}
```

7.7 帶權重排程

```
// 帶權區間排程 Weighted Interval Scheduling
// dp[i] = max(dp[i-1], w[i] + dp[h[i]])
// h[i]: 最後一個結束時間 < l[i] 的任務 (用二分查找)
// 時間複雜度 O(n log n)
// 每個任務 l <= r；可不選任務，函式會依 r 排序 p。
struct Project { long long l, r, w; };
bool cmp(const Project& a, const Project& b) {
  return a.r < b.r;
}
long long weightedInterval(vector<Project>& p) {
  int n = p.size();
  sort(p.begin(), p.end(), cmp);
  vector<long long> dp(n + 1), endTime;
  for (auto& x : p) endTime.push_back(x.r);
  for (int i = 0; i < n; ++i) {
    int j = lower_bound(endTime.begin(),
        endTime.begin() + i, p[i].l) - endTime.begin();
    dp[i + 1] = max(dp[i], p[i].w + dp[j]);
  }
  return dp[n];
}
```

8 Divide and conquer

8.1 逆序數對

```
// 依 second 計逆序數對，並將 [l, r] 穩定排序。
// 回傳 long long；暫存陣列只配置一次，O(n log n)。
long long mergeSort(vector<pair<int, int>>& v,
    vector<pair<int, int>>& tmp, int l, int r) {
  if (l >= r) return 0;
  int mid = l + (r - l) / 2;
  long long count = mergeSort(v, tmp, l, mid)
                  + mergeSort(v, tmp, mid + 1, r);
  int i = l, j = mid + 1;
  for (int k = l; k <= r; ++k) {
    if (j > r || (i <= mid &&
                  v[i].second <= v[j].second))
      tmp[k] = v[i++];
    else tmp[k] = v[j++], count += mid - i + 1;
  }
  copy(tmp.begin() + l, tmp.begin() + r + 1,
       v.begin() + l);
  return count;
}
long long mergeSort(vector<pair<int, int>>& v,
```

```
                    int l, int r) {
  vector<pair<int, int>> tmp(v.size());
  return mergeSort(v, tmp, l, r);
}

signed main() {
  int n;
  cin >> n;
  vector<pair<int, int>> arr(n);
  for (int i = 0; i < n; ++i) {
    arr[i].first = i;
    cin >> arr[i].second;
  }
  cout << mergeSort(arr, 0, n - 1) << '\n';
}
```

8.2 Mo's algorithm

```
// time complexity: n * sqrt(q) * O(p)
// O(p) 為 add, remove 的時間複雜度
// 若知道 [l, r] 的答案 需要快速知道 [l
    - 1, r], [l + 1, r], [l, r - 1], [l, r + 1] 的答案

int n, q, k, l = 0, r = 0;

array queries = 詢問們;
type ans; //目前答案
void add(type v){/*...*/} //增加一個數字，算新答案
void remove(type v){/*...*/} //移除一個數字，算新答案

vector<tuple<int, int, int, int>> queries(q);
k = sqrt(n);
for(int i = 0; i < q; i++){
  int l, r;
  cin >> l >> r;
  queries[i] = {l / k, r, l, i};
  // 先對 l 的塊，再對 r 排序
}

sort(queries.begin(), queries.end());

add(a[0]);

for(int i = 0; i < q; i++){
  auto [_, rp, lp, id] = queries[i];
  lp--; rp--;
  while(l > lp) add(a[--l]);
  while(l < lp) remove(a[l++]);
  while(r < rp) add(a[++r]);
  while(r > rp) remove(a[r--]);
  ans_v[id] = ans;
}
```

9 Tree

9.1 樹直徑

```
int d1[200005], d2[200005], ans;

void dfs(int now, int fa, vector<vector<int>> &graph){
  for(auto i: graph[now]){
    if(i != fa){
      dfs(i, now, graph);
      if(d1[i] + 1 > d1[now]){
        d2[now] = d1[now];
        d1[now] = d1[i] + 1;
      }
      else if(d1[i] + 1 > d2[now]){
        d2[now] = d1[i] + 1;
      }
    }
  }
  ans = max(ans, d1[now] + d2[now]);
}

signed main()
{
  int n;
  cin >> n;
  vector<vector<int>> graph(n + 1);
  for(int i = 0; i < n - 1; i++){
    int a, b;
    cin >> a >> b;
    graph[a].push_back(b);
    graph[b].push_back(a);
  }
  dfs(1, 0, graph);
  cout << ans << '\n';
}
```

9.2 LCA

```
// n 為點數，graph 由父節點往子節點建有向邊
// graph 要 resize；1-base，n <= 200000。
// build_lca 可重建；用堆疊走訪，避免深鏈遞迴。
int n, q;
int fa[18][200001], dep[200001];
vector<vector<int>> graph;

void dfs(int root, int lst){
  fa[0][root] = lst;
  vector<int> st{root};
  while(!st.empty()){
    int u = st.back(); st.pop_back();
    for(int v: graph[u]){
      fa[0][v] = u, dep[v] = dep[u] + 1;
      st.push_back(v);
    }
  }
}

void build_lca(int root){
  dep[root] = 1;
  dfs(root, root);
  for(int i = 1; i < 18; i++)
    for(int j = 1; j <= n; j++)
      fa[i][j] = fa[i - 1][fa[i - 1][j]];
}

int lca(int a, int b){
  // 預設 a 比 b 淺，讓 a 和 b 跳到同一個地方
  if(dep[a] > dep[b]) swap(a, b);
  int step = dep[b] - dep[a];
  for(int i = 0; i < 18; i++)
    if(step >> i & 1) b = fa[i][b];
  if(a == b) return a;
  for(int i = 17; i >= 0; i--)
    if(fa[i][a] != fa[i][b])
      a = fa[i][a], b = fa[i][b];
  return fa[0][a];
}
```

9.3 樹壓平

```
//紀錄in & out
vector<int> Arr;
vector<int> In, Out;
void dfs(int u) {
  Arr.push_back(u);
  In[u] = Arr.size() - 1;
  for (auto v : Tree[u]) {
    if (v == parent[u])
      continue;
    parent[v] = u;
    dfs(v);
  }
  Out[u] = Arr.size() - 1;
}

//進去出來都紀錄
vector<int> Arr;
void dfs(int u) {
  Arr.push_back(u);
  for (auto v : Tree[u]) {
    if (v == parent[u])
      continue;
    parent[v] = u;
    dfs(v);
  }
  Arr.push_back(u);
}

//用Treap紀錄
Treap *root = nullptr;
vector<Treap *> In, Out;
void dfs(int u) {
  In[u] = new Treap(cost[u]);
  root = merge(root, In[u]);
  for (auto v : Tree[u]) {
    if (v == parent[u])
```

```
      continue;
    parent[v] = u;
    dfs(v);
  }
  Out[u] = new Treap(0);
  root = merge(root, Out[u]);
}
//Treap紀錄Parent
struct Treap {
  Treap *lc = nullptr, *rc = nullptr;
  Treap *pa = nullptr;
  unsigned pri, size;
  long long Val, Sum;
  Treap(int Val):
    pri(rand()), size(1),
    Val(Val), Sum(Val) {}
  void pull();
};

void Treap::pull() {
  size = 1;
  Sum = Val;
  pa = nullptr;
  if (lc) {
    size += lc->size;
    Sum += lc->Sum;
    lc->pa = this;
  }
  if (rc) {
    size += rc->size;
    Sum += rc->Sum;
    rc->pa = this;
  }
}
//找出節點在中序的編號
size_t getIdx(Treap *x) {
  assert(x);
  size_t Idx = 0;
  for (Treap *child = x->rc; x;) {
    if (child == x->rc)
      Idx += 1 + size(x->lc);
    child = x;
    x = x->pa;
  }
  return Idx;
}
//切出想要的東西
void move(Treap *&root, int a, int b) {
  size_t a_in = getIdx(In[a]), a_out = getIdx(Out[a]);
  auto [L, tmp] = splitK(root, a_in - 1);
  auto [tree_a, R] = splitK(tmp, a_out - a_in + 1);
  root = merge(L, R);
  tie(L, R) = splitK(root, getIdx(In[b]));
  root = merge(L, merge(tree_a, R));
}
```

9.4 樹鏈剖分

```
// 從 top 的深度較低的那個節點開始跳
// 跳到同一條鏈上 (top 相同)
// 一邊跳一邊更新線段樹
// in 是 dfn

struct HLD {
    // 0-base
    int n, cur;
    vector<int> siz, top, dep, parent, in, out, seq;
    vector<vector<int>> adj;
    HLD(int n) : n(n), cur(0) {
        siz.resize(n); top.resize(n); dep.resize(n);
        parent.resize(n); in.resize(n); out.resize(n);
        seq.resize(n); adj.assign(n, {});
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void work(int rt = 0) {
        top[rt] = rt;
        dep[rt] = 0;
        parent[rt] = -1;
        dfs1(rt); dfs2(rt);
    }
    void dfs1(int u) {
        if (parent[u] != -1)
            adj[u].erase(find(adj
                [u].begin(), adj[u].end(), parent[u]));
        siz[u] = 1;
        for (auto &v : adj[u]) {
            parent[v] = u, dep[v] = dep[u] + 1;
            dfs1(v);
            siz[u] += siz[v];
            if (siz[v] > siz[adj[u][0]]) {
                swap(v, adj[u][0]);
            } // 讓 adj[u][0] 是重子節點
        }
    }
    void dfs2(int u) {
        in[u] = cur++;
        seq[in[u]] = u; // dfn 對應的編號
        for (auto v : adj[u]) {
            top[v] = v == adj[u][0] ? top[u] : v;
            dfs2(v);
        }
        out[u] = cur;
    }
    int lca(int u, int v) {
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]]) {
                u = parent[top[u]];
            } else {
                v = parent[top[v]];
            }
        }
        return dep[u] < dep[v] ? u : v;
    }
    int dist(int u, int v) {
        return dep[u] + dep[v] - 2 * dep[lca(u, v)];
    }
    int jump(int u, int k) {
        if (dep[u] < k) return -1;
        int d = dep[u] - k;
        while (dep[top[u]] > d) u = parent[top[u]];
        return seq[in[u] - dep[u] + d];
    }
    bool isAncester(int u, int v) {
        return in[u] <= in[v] && in[v] < out[u];
    }
    int rootedParent(int rt, int v) {
        if (rt == v) return rt;
        if (!isAncester(v, rt)) return parent[v];
        auto it = upper_bound
            (adj[v].begin(), adj[v].end(), rt,
            [&](int x, int y) {
                return in[x] < in[y];
            }) - 1;
        return *it;
    }
    int rootedSize(int rt, int v) {
        if (rt == v) return n;
        if (!isAncester(v, rt)) return siz[v];
        return n - siz[rootedParent(rt, v)];
    }
    int rootedLca(int rt, int a, int b) {
        return lca(rt, a) ^ lca(a, b) ^ lca(b, rt);
    }
};

// example 找兩個子節點間的 max 以及改值

signed main() {
  int n, q;
  cin >> n >> q;

  vector<int> tp(n);
  for (int i = 0; i < n; i++) cin >> tp[i];

  HLD t(n);
  for (int i = 0; i < n - 1; i++) {
    int u, v;
    cin >> u >> v;
    --u; --v;
    t.addEdge(u, v);
  }

  t.work(0); // 以 0 (節點1) 為根

  // 將點值依照 dfn 映射到線段樹底層
  for (int i = 0; i < n; i++) {
    a[t.in[i]] = tp[i];
```

```
  }
  build(1, 0, n - 1); // 根 id = 1

  auto path_max = [&](int u, int v) {
    int res = LLONG_MIN;
    // 往上跳鏈 邊跳邊維護答案
    while (t.top[u] != t.top[v]) {
      if (t
          .dep[t.top[u]] < t.dep[t.top[v]]) swap(u, v);
      res = max(res,
          query(1, 0, n - 1, t.in[t.top[u]], t.in[u]));
      u = t.parent[t.top[u]];
    }
    int l = t.in[u], r = t.in[v];
    if (l > r) swap(l, r);
    res = max(res, query(1, 0, n - 1, l, r));
    return res;
  };

  while (q--) {
    int mode;
    cin >> mode;
    if (mode == 1) {
      int s, x;
      cin >> s >> x;
      --s;
      modify(1, 0, n - 1, t.in[s], x);
    } else if (mode == 2) {
      int a, b;
      cin >> a >> b;
      --a; --b;
      cout << path_max(a, b) << '\n';
    }
  }
}
```

10 Else

10.1 Big Number

```
string Add(const string &a, const string &b) {
    int n
        = a.length() - 1, m = b.length() - 1, car = 0;
    string res;
    while (n >= 0 || m >= 0 || car) {
        int x = (n >= 0 ? a[n] -
            '0' : 0) + (m >= 0 ? b[m] - '0' : 0) + car;
        res += (x % 10) + '0';
        car = x / 10;
        n--, m--;
    }
    while (res.length() > 1 && res.back() == '0') {
        res.pop_back();
    }
    reverse(res.begin(), res.end());
    return res;
}
string Minus(const string &a, const string &b) {
    // Assume a >= b
    int n
        = a.length() - 1, m = b.length() - 1, bor = 0;
    string res;
    while (n >= 0) {
        int x = a[n] - '0' - bor;
        int y = m >= 0 ? b[m] - '0' : 0;
        bor = 0;
        if (x < y) {
            x += 10;
            bor = 1;
        }
        res += x - y + '0';
        n--, m--;
    }
    while (res.length() > 1 && res.back() == '0') {
        res.pop_back();
    }
    reverse(res.begin(), res.end());
    return res;
}
string Multiple(const string &a, const string &b) {
    string res = "0";
    int n = a.length() - 1, m = b.length() - 1;
    for (int i = m; i >= 0; i--) {
        string add;
        int car = 0;
        for (int j = n; j >= 0 || car; j--) {
            int x = (j >= 0
                ? a[j] - '0' : 0) * (b[i] - '0') + car;
            add += (x % 10) + '0';
            car = x / 10;
        }
        while (add.length() > 1 && add.back() == '0') {
            add.pop_back();
        }
        reverse(add.begin(), add.end());
        res = Add(res, add + string(m - i, '0'));
    }
    return res;
}
```

10.2 Tenary Search

```
// return the maximum of $f(x)$ in $[l, r]$
// f 在最大值區間外嚴格增／減；EPS > 0、r-l 有限。
double ternary_search(double l, double r) {
  while (r - l > EPS) {
    double m1 = l + (r - l) / 3;
    double m2 = r - (r - l) / 3;
    if (m1 == l || m2 == r) break;
    double f1 = f(m1), f2 = f(m2);
    if (f1 < f2) l = m1;
    else r = m2;
  }
  return max(f(l), f(r));
}

// return the maximum of $f(x)$ in $(l, r]$
// 整數版回傳「位置」；l < r，r-l 須放得進 int。
// f 在最大值區間外嚴格增／減；相同最大值取最右。
int ternary_search(int l, int r) {
  while (r - l > 1) {
    int mid = l + (r - l) / 2;
    if (f(mid) > f(mid + 1)) r = mid;
    else l = mid;
  }
  return r;
}
```

10.3 對拍

```
#include <iostream>
using namespace std;
int main() {
    for (int T=1;;T++){
        if (system("./random > test.in")) {
            cout << "random RE on " << T << '\n';
            return 0;
        }
        if (system("./sol < test.in > test.out")) {
            cout << "sol RE on " << T << '\n';
            return 0;
        }
        if (system("./bf < test.in > test.ans")) {
            cout << "bf RE on " << T << '\n';
            return 0;
        }
        if (system("diff -Z test.out test.ans")) {
            cout << "WA on " << T << '\n';
            return 0;
        } else {
            cout << "AC on " << T << '\n';
        }
    }
}
```

10.4 Random Generator

```
#include <iostream>
#include <random>
int main() {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);
    std::cout << "Get Rand: " << distrib(gen) << '\n';
}
```